

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 1 czerwca 2026

Spis treści

Streszczenie	6
1 Wprowadzenie	7
1.1 Motywacja i cel pracy	7
1.2 Zakres pracy	7
1.3 Struktura pracy	8
2 Analiza wymaga i przegląd rozwiązań	9
2.1 Wymagania funkcjonalne	9
2.2 Wymagania нефункционалне	10
2.3 Przegląd istniejących rozwiązań	11
2.3.1 OctoPrint	11
2.3.2 Mainsail oraz Fluidd	11
2.3.3 Platformy komercyjne	11
2.3.4 Wnioski końcowe	12
3 Wybór technologii	13
3.1 Język programowania oraz środowisko uruchomieniowe	13
3.1.1 TypeScript oraz Node.js	13
3.1.2 Python	13
3.2 Framework webowy — Express.js	13
3.3 Baza danych — Azure Cosmos DB	14
3.4 Repozytorium obiektowe — Azure Blob Storage	14
3.5 Broker komunikatów — Azure Service Bus	14
3.6 Sterowanie urządzeniem — Azure IoT Hub	15
3.7 Interfejs użytkownika — React oraz Vite	15
3.8 Konteneryzacja — Docker oraz Docker Compose	16
3.9 Automatyzacja infrastruktury chmurowej — Terraform	16
3.10 środowisko hostingowe warstwy serwerowej — Oracle Cloud Infrastructure	17
3.11 CAD - SolidWorks	17
3.12 Porównanie wybranych alternatyw	17

4	Architektura systemu	20
4.1	Ogólna struktura	20
4.2	Diagram architektury	20
4.3	Model danych	22
4.3.1	Użytkownik (kontener users)	23
4.3.2	Zadanie do wydruku (kontener jobs)	24
4.3.3	Token odwołania (kontener refresh-tokens)	24
4.4	Cykł życia zadania do wydruku	25
4.5	Bezpieczeństwo i uwierzytelnianie	26
5	Implementacja serwisów warstwy serwerowej	28
5.1	Auth Service	28
5.1.1	Opis ogólny	28
5.1.2	Punkty końcowe API	28
5.1.3	Kluczowe fragmenty implementacji	29
5.1.4	Weryfikacja adresu poczty elektronicznej	31
5.2	User Service	31
5.2.1	Opis ogólny	31
5.2.2	Punkty końcowe API	32
5.2.3	Mechanizm autoryzacji dla	33
5.3	Files Service	35
5.3.1	Opis ogólny	35
5.3.2	Procedura przetwarzania i przesyłania plików	35
5.4	Queue Service	37
5.4.1	Opis ogólny	37
5.4.2	Mechanizm nasychiwania kolejki	37
5.4.3	Punkty końcowe HTTP API	39
5.5	Printer Service	40
5.5.1	Opis ogólny	40
5.5.2	Mechanizm harmonogramowania	40
5.5.3	Monitor gotowości urządzenia	42
5.5.4	Odbiornik telemetry	42
5.5.5	Punkty końcowe HTTP API	43

5.6	Video Service	44
5.6.1	Opis ogólny	44
5.6.2	Punkty kocowe API	45
6	Implementacja warstwy prezentacji	46
6.1	Struktura architektoniczna aplikacji klienckiej	46
6.2	Scentralizowany klient API	47
6.3	Mechanizm routingu i ochrona tras	48
6.4	Ekrany aplikacji klienckiej	49
6.4.1	Strona główna systemu	49
6.4.2	Panel użytkownika	50
6.4.3	Przesyłanie i planowanie zadań	50
6.4.4	Kontrola procesu drukowania	51
6.4.5	Panel administracyjny	52
6.5	Internacjonalizacja aplikacji	53
6.6	Zarządzanie motywami graficznymi interfejsu	53
7	Agent sprzętowy — AddiPi Agent	55
7.1	Opis ogólny	55
7.2	Struktura obiektowa i architektura klas	55
7.3	Połączenie z IoT Hub	56
7.4	Obsługiwane metody bezpośrednie	56
7.5	Przebieg obsługi metody bezpośredniej startPrint	57
7.6	Transmisja danych telemetrycznych	58
7.7	Niskopoziomowa komunikacja z serwerem OctoPrint	58
8	Projekt mechaniczny obudowy i pozycjonowania zestawu	60
8.1	Cel i zakres projektu mechanicznego	60
8.2	Moduł CASE — obudowa Raspberry Pi z kamerą	60
8.2.1	Podstawa projektowa	60
8.2.2	Zakres modyfikacji geometrycznych	61
8.2.3	Integracja strukturalna i mocowanie modułu kamery brzegowej	61
8.2.4	Struktura plików i organizacja zasobów modułu CASE	62
8.3	Moduł STAND — regulowany mechanizm pozycjonujący	62

8.4	Zoenie kompletnego modelu geometrycznego	63
8.5	Parametry wydruku	64
8.6	Procedura montau zestawu mechanicznego	65
9	Konfiguracja Raspberry Pi	66
9.1	Opis rodowiska	66
9.2	Instalacja agenta	66
9.3	Automatyzacja uruchamiania za pomoc systemd	67
9.4	Udostpnianie strumienia wideo przez Cloudflare Tunnel	68
9.4.1	Instalacja narzdzia cloudflared	68
9.4.2	Skrypt automatyzacji publikacji adresu sieciowego	69
9.4.3	Automatyzacja odnawiania adresu w harmonogramie cron	70
9.5	Konfiguracja harmonogramu zada crontab	70
9.5.1	Uzasadnienie zastosowanych parametrów	71
9.6	Problem dostpnoci sieci podczas uruchamiania systemu	71
9.7	Podsumowanie konfiguracji wza brzegowego	72
10	Wdroenie i infrastruktura produkcyjna	74
10.1	Hybrydowa strategia architektoniczna	74
10.2	Zarzdzane usugi integracji danych i komunikacji chmurowej	74
10.3	Automatyzacja orkiestracji chmurowej — Terraform	75
10.4	Specyfikacja konteneryzacji i optymalizacja obrazów	76
10.5	Automatyzacja procesów CI/CD	77
10.6	Orkiestracja kontenerowa warstwy serwerowej — Docker Compose	78
10.7	Zarzdzanie konfiguracji i sekretami przez zmienne rodowiskowe	80
10.8	Wdroenie warstwy prezentacji oraz usugi wideo	80
10.9	Wdroenie komponentu brzegowego	81
11	Podsumowanie i wnioski kocowe	82
11.1	Osignite rezultaty i wkad inzynierski	82
11.2	Analiza napotkanych trudnoci projektowo-implementacyjnych	84
11.3	Ograniczenia przyjtego rozwizania	86
11.4	Perspektywy i potencjalne kierunki dalszego rozwoju systemu	86
11.5	Wnioski kocowe	88

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz walidację rozproszonego systemu zarządzania procesami druku trójwymiarowego o nazwie *AddiPi*, dedykowanego dla akademickich laboratoriów studenckich. Opracowane rozwiązanie umożliwi użytkownikom zdalne przysyłanie plików z kodem sterującym (G-code), harmonogramowanie oraz monitorowanie stanu realizacji zadań w czasie rzeczywistym. Administratorom systemu udostępniono natomiast moduł pełnej kontroli nad kolejką produkcyjną, urządzeniami oraz profilami użytkowników.

Architektura systemu została oparta na paradygmacie mikroserwisowym. W strukturze warstwy serwerowej wyodrębniono sze niezależnych komponentów: usług uwierzytelniania (*Auth Service*), zarządzania profilami (*User Service*), obsługi plików (*Files Service*), orkiestracji kolejki zadań (*Queue Service*), sterowania urządzeniem (*Printer Service*) oraz agenta sprzętowego (*AddiPi Agent*) uruchamianego na platformie Raspberry Pi. Warstwę prezentacji stanowi aplikacja kliencka (frontendowa) zbudowana z wykorzystaniem biblioteki React, która uzupełnia dedykowany serwis wideo (*Video Service*) odpowiedzialny za dystrybucję strumienia wizyjnego.

Komunikacja wewnątrz systemu realizowana jest hybrydowo — za pomocą synchronicznego protokołu HTTP (architektura REST API) oraz asynchronicznej, chmurowej kolejki *Azure Service Bus*. Dane aplikacyjne są utrwalane w wielomodelowej bazie danych *Azure Cosmos DB*, a pliki G-code w repozytorium obiektowym *Azure Blob Storage*. Zarządzanie i sterowanie fizycznymi drukarkami 3D odbywa się za pośrednictwem usługi *Azure IoT Hub* przy użyciu mechanizmu metod bezpośrednich (ang. *Direct Methods*) wywoływanych na mikrokomputerze, na którym osadzono oprogramowanie serwerowe *OctoPrint*.

Poszczególne serwisy zaimplementowano w językach TypeScript (rodowisko Node.js) oraz Python. Proces wdrożenia środowiska oparto na technologii konteneryzacji Docker, koordynowanej za pomocą narzędzia Docker Compose.

Słowa kluczowe: mikroserwisy, drukowanie 3D, OctoPrint, Raspberry Pi, Azure IoT Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, systemy rozproszone.

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowi obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe [1]. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym — ich liczba w danej jednostce jest zwykle ograniczona, a czas wytwarzania pojedynczego modelu fizycznego może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne wyzwania logistyczne i organizacyjne, takie jak konieczność rezerwowania terminów, brak przejrzystości w strukturze kolejki zadań oraz brak możliwości zdalnego monitorowania stanu operacyjnego maszyn.

Głównym celem niniejszej pracy jest zaprojektowanie, implementacja oraz walidacja rozproszonego systemu *AddiPi*, którego zadaniem jest pełna automatyzacja i orkiestracja procesów zarządzania drukarkami 3D w środowisku laboratorium studenckiego. Opracowane oprogramowanie udostępnia użytkownikom funkcjonalności w zakresie:

- asynchronicznego przesyłania plików z kodem sterującym (G-code) za pośrednictwem aplikacji klienckiej,
- elastycznego harmonogramowania procesów druku na określony termin lub lokowania ich w ogólnej kolejce systemu,
- śledzenia bieżących metryk postępu prac w czasie rzeczywistym wraz z bezpośrednim podglądem wizyjnym,
- automatycznego otrzymywania powiadomień o pomyślnym zakończeniu lub awarii procesu produkcyjnego.

Personelowi administracyjnemu laboratorium system zapewnia dedykowane narzędzia do zarządzania profilami użytkowników i ich uprawnieniami, audytu historii wszystkich operacji oraz zdalnego sterowania infrastrukturą sprzętową i priorytetami kolejki.

1.2 Zakres pracy

Zakres rzeczowy zrealizowanego projektu inżynierskiego obejmuje:

1. przeprowadzenie analizy wymaga funkcjonalnych i нефункциональных oraz wybór technologii,
2. opracowanie projektu rozproszonej architektury mikroserwisowej,
3. implementację komponentów serwerowych (backendowych) oraz aplikacji klienckiej (frontendowej),
4. konfigurację i integrację dedykowanej infrastruktury chmurowej w modelu PaaS (*Microsoft Azure*),
5. konteneryzacji oraz automatyzacji procesu wdrożenia systemu przy użyciu narzędzi *Docker* oraz *Docker Compose*,
6. przeprowadzenie testów weryfikacyjnych i walidacji poprawności działania całego ekosystemu.

1.3 Struktura pracy

Niniejsza praca dyplomowa została podzielona na logiczne segmenty, których zawarto odzwierciedla kolejne etapy realizacji projektu. Rozdział 2 zawiera szczegółów analiz wymaga funkcjonalnych i нефункциональных oraz krytyczny przegląd istniejących rozwiązań rynkowych. W rozdziale 3 omówiono wybrane technologie programistyczne i chmurowe wraz z technicznym uzasadnieniem podjętych decyzji projektowych. Projekt samej architektury rozproszonej oraz przepływu danych w systemie zostały opisane w rozdziale 4.

Szczegółowa implementacja poszczególnych komponentów programowych, z uwzględnieniem warstwy serwerowej, klienckiej oraz komponentów pomocniczych, stanowi treść rozdziałów od 5 do 7. Z kolei w rozdziale 8 przedstawiono proces projektowania mechanicznego i trójwymiarowego modelowania obudowy urządzenia w programie *SolidWorks*. Specyfika konfiguracji mikrokomputera sterującego, automatyzacja zadań systemowych przez mechanizm *cron* oraz konfiguracja bezpiecznego tunelu sieciowego *Cloudflare* zostały przybliżone w rozdziale 9.

Końcowe etapy prac, obejmujące orkiestrację infrastruktury chmurowej *Microsoft Azure* oraz procesy konteneryzacji środowiska, opisano w rozdziale 10. Całość pracy kończy się podsumowaniem oraz wnioskami końcowymi, zawartymi w rozdziale 11.

2 Analiza wymaga i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie szczegółowej analizy specyfikacji oraz potrzeb akademickiego laboratorium studenckiego sformuowano następujące wymagania funkcjonalne, z podziałem na role użytkownika standardowego oraz administratora systemu.

W obszarze kont użytkowników (rola standardowa):

- rejestracja konta powiązana z dwuetapową weryfikacją adresu e-mail, ograniczoną wyłącznie do domeny akademickiej @uwr.edu.pl,
- bezpieczne uwierzytelnianie i autoryzacja z wykorzystaniem mechanizmu podwójnych tokenów JWT (*access token* oraz *refresh token*) [2],
- asynchroniczne przesyłanie plików z kodem sterującym (G-code) o maksymalnym rozmiarze pojedynczego wolumenu do 50 MB i rozszerzeniu .gcode,
- elastyczne harmonogramowanie procesów wytwórczych na wybrany termin kalendarzowy lub lokowanie ich w ogólnej kolejce systemu,
- przeglądanie historii własnych zleceń produkcyjnych z opcją dynamicznego filtrowania według stanów maszyny,
- monitorowanie parametrów środowiskowych i postępu procesu druku 3D w czasie rzeczywistym (weryfikacja paska postępu, temperatury dyszy oraz prędkości roboczej, a także podgląd wizyjny z kamery),
- możliwość manualnego anulowania lub ponownego inicjowania własnych zadań.

W obszarze administracji i nadzoru (rola administratora):

- przegląd rejestru kont oraz zarządzanie profilami wszystkich użytkowników systemu,
- dynamiczna modyfikacja poziomu uprawnień i ról użytkowników (nadawanie statusu *user* lub *admin*),
- kompleksowe zarządzanie globalną kolejką oraz historią wszystkich zarejestrowanych zadań drukowania,

- zdalne potwierdzanie fizycznej i operacyjnej gotowości urzduzenia do podjcia kolejnego cyklu produkcyjnego,
- dostp do zagregowanych metryk systemowych, obejmujcych m.in. statystyki zada z podziaem na statusy.

2.2 Wymagania niefunkcjonalne

W celu zapewnienia odpowiedniej jakoci, wydajności oraz stabilności dziaania rozproszonej platformy informatycznej, okrełono nastpujce wymagania jakociowe (niefunkcjonalne):

- **Skalowalno** [3] — architektura rozproszona powinna umoliwia bezproblemow integracj i rejestracj kolejnych fizycznych urzduze drukujcych bez konieczności modyfikacji lub przebudowy rdzenia systemu.
- **Niezawodno i tolerancja bdów** — asynchroniczne przetwarzanie komunikatów z wykorzystaniem magistrali powinno gwarantowa, e chwilowa niedostpno komponentów nie spowoduje trwaej utraty zlecenia druku ani niespójności stanów w bazie danych.
- **Bezpieczestwo danych** — autoryzacja dostpu do interfejsów programistycznych REST API realizowana za pomoc kryptograficznie podpisanych tokenów JWT [2], natomiast hasa uytkowników bezwzglđnie utrwalane w bazie danych w postaci kryptograficznych skrótów (ang. *hashes*) wygenerowanych algorytmem *bcrypt* [4].
- **Responsywno interfejsu** — warstwa prezentacji (aplikacja frontendowa) musi zosta zaprojektowana zgodnie z paradygmatem RWD (ang. *Responsive Web Design*), zapewniajc pen ergonomi i poprawno wywietlania zarówno na urzduzeniach stacjonarnych, jak i mobilnych.
- **Wydajno czasowa** — czas odpowiedzi punktów kocowych API na zapytania synchroniczne nie powinien przekracza 2 sekund w warunkach standardowego, nominalnego obciaenia infrastruktury.
- **Przenono i konteneryzacja** — wszystkie komponenty, mikroserwisy oraz bazy danych wchodzcze w skad systemu musz zosta spakowane w niezalene kontenery rodowiska *Docker*, umoliwiajc pene uruchomienie caego ekosystemu za pomoc jednego polecenia skryptowego (*Docker Compose*).

2.3 Przegląd istniejących rozwiązań

2.3.1 OctoPrint

Oprogramowanie *OctoPrint* [5] stanowi obecnie jedno z najpopularniejszych rozwiązań typu *open-source* dedykowanych do zdalnego zarządzania i monitorowania pojedynczych drukarek 3D. System ten, uruchamiany najczęściej jako usługa serwerowa w środowisku Linux na mikrokomputerze Raspberry Pi, udostępnia responsywny interfejs webowy, programistyczny interfejs REST API oraz rozbudowany ekosystem wtyczek (ang. *plugins*). Umożliwia on bezporównanie kontrolowanie parametrów mechanicznych i termicznych drukarki, wizualizację kodu sterującego, podgląd wideo z kamery oraz analiz postępu prac.

Głównym ograniczeniem platformy *OctoPrint* jest jednak jej monolityczna architektura, zaprojektowana z myślą o pojedynczym użytkowniku zarządzającym wyłącznie jedną maszyną. Oprogramowanie to nie oferuje natywnych mechanizmów globalnego kolejkowania zadań dla wielu niezależnych kont, zaawansowanego systemu kontroli dostępu opartego na rolach (RBAC) ani integracji z infrastrukturą chmurową. Z tego względu w projekcie *AddiPi* system *OctoPrint* został wykorzystany wyłącznie w charakterze lokalnego, niskopoziomowego sterownika sprzętowego, nad którym nadbudowano rozproszoną warstwę orkiestracji i zarządzania.

2.3.2 Mainsail oraz Fluidd

Ekosystemy *Mainsail* oraz *Fluidd* to nowoczesne, wysoce zoptymalizowane interfejsy webowe współpracujące bezpośrednio z oprogramowaniem układowym (firmware) *Klipper*. Charakteryzują się niskim narzutem wydajnościowym oraz zaawansowanymi algorytmami kompensacji drgań maszyn. Podobnie jednak jak w przypadku oprogramowania *OctoPrint*, ich architektura została zorientowana na scenariusz jednorodny (jeden operator — jedno urządzenie) i nie przewiduje mechanizmów współdzielenia zasobów sprzętowych ani wielodostępczej orkiestracji kolejki zadań.

2.3.3 Platformy komercyjne

Rozwiązania takie jak *3DprinterOS*, *Polar Cloud* czy *Ultimaker Digital Factory* to komercyjne systemy klasy *fleet management*, przeznaczone do scentralizowanego zarządzania rozproszoną flotą drukarek trójwymiarowych w sektorze przemysłowym oraz edukacyjnym.

Platformy te udostępniają zaawansowane moduły analityczne i kolejki zadań, jednak ich implementacja wiąże się z wysokimi, cyklicznymi kosztami subskrypcyjnymi. Dodatkowo są to systemy zamknięte, działające w modelu SaaS (*Software as a Service*), co uniemożliwia pełną kontrolę nad integralnością i lokalizacją przetwarzanych danych — w specyficznych warunkach laboratoriów akademickich stanowi to istotną barierę wdrożeń.

2.3.4 Wnioski końcowe

Przeprowadzona analiza wykazała, że żadne z dostępnych na rynku rozwiązań o otwartym kodzie źródłowym nie czy w sobie wielodostępnego systemu uwierzytelniania, asynchronicznego kolejkowania zadań z funkcją planowania kalendarzowego, bezpośredniej integracji z chmurą obliczeniową oraz sterowania bazującego na paradygmacie Internetu Rzeczy (IoT). Projekt *AddiPi* skutecznie wypełnia te luki technologiczne. Stanowi on autonomiczne, bezpieczne i wysoce elastyczne środowisko, które pozwala na pełną niezależność infrastrukturalną przy jednoczesnym zachowaniu niskich kosztów utrzymania systemu.

3 Wybór technologii

3.1 Jzyk programowania oraz rodowisko uruchomieniowe

3.1.1 TypeScript oraz Node.js

Wikszo serwisów warstwy serwerowej oraz aplikacj klienck zaimplementowano w jzyku TypeScript [6], bdcym nadzbiorem jzyka JavaScript, który wprowadza statyczne typowanie. Wybór ten znaczco uatwia utrzymanie rozbudowanych baz kodu [7] poprzez wykrywanie bdów na etapie kompilacji, zaawansowane wsparcie rodowisk programistycznych (IDE) oraz samodokumentowanie struktury danych za pomoc typów.

Platform Node.js [8] wybrano ze wzgldu na asynchroniczny, nieblokujcy model wejcia-wyjcia. Rozwizanie to charakteryzuje si wysok wydajnoci w systemach obsugujcych liczne, równolege poczenia HTTP oraz intensywn komunikacj z usugami zewntrznymi, takimi jak bazy danych, kolejki komunikatów czy usuga *Azure IoT Hub*.

3.1.2 Python

Agent dziaajcy na platformie Raspberry Pi zosta zaimplementowany w jzyku Python [9]. Biblioteki *azure-iot-device* oraz *azure-storage-blob* s w tym rodowisku powszechnie dostpne i szczegóowo udokumentowane. Python stanowi standardowe rozwizanie w przypadku aplikacji uruchamianych na systemach wbudowanych oraz mikrokomputerach jednoukadowych (ang. *Single-Board Computer*, SBC).

3.2 Framework webowy — Express.js

Wszystkie serwisy warstwy Node.js [8] wykorzystuj framework Express.js [10] w wersji stabilnej 5.0. Rozwizanie to charakteryzuje si minimalistyczn architektur i niskim narzutem wydajnościowym. Framework nie narzuca sztywnej struktury katalogów (ang. *unopinionated framework*), co pozwala na elastyczne dostosowanie architektury kadego z mikroservisów do jego specyficznych wymaga. Express.js zapewnia równie ustandaryzowane mechanizmy obsugi da poredniczcych (ang. *middleware*), routingu oraz zarzdzania bdami.

3.3 Baza danych — Azure Cosmos DB

Azure Cosmos DB [11] to w pełni zarządzana, wielomodelowa baza danych typu NoSQL, oferująca globalną dystrybucję danych, elastyczne partycjonowanie oraz natywny obsługa zróżnicowanych interfejsów programistycznych (SQL, MongoDB, Cassandra, Gremlin, Table). W projekcie *AddiPi* wykorzystano interfejs API SQL (Core), który umożliwia wykonywanie zapytań w standardzie zbliżonym do języka SQL na dokumentach w formacie JSON.

Wybór tej technologii został podyktowany następującymi czynnikami:

- **Model usługowy PaaS (Platform as a Service)** — pełne zarządzanie infrastrukturą przez dostawcę chmurowego eliminuje konieczność administracji serwerem bazy danych,
- **Elastyczny schemat danych** — poszczególne mikroserwisy przechowują obiekty o zróżnicowanej, heterogenicznej strukturze w ramach jednej bazy danych (*addi pi*) [3], z podziałem na dedykowane kontenery (*users*, *jobs*, *refresh-tokens*),
- **Natywna integracja** — bezszwowe współdziałanie z pozostałymi komponentami ekosystemu chmurowego *Microsoft Azure*,
- **Automatyczne indeksowanie** — domyślne indeksowanie wszystkich pól w strukturze dokumentu, co optymalizuje czas wykonywania zapytań.

3.4 Repozytorium obiektowe — Azure Blob Storage

Pliki G-code przesłane przez użytkowników są przechowywane w usłudze repozytorium obiektowego *Azure Blob Storage* [12] w ramach dedykowanego kontenera o nazwie *gcode*. Usługa ta zapewnia niemal nieograniczoną przestrzeń magazynową, wysoki poziom dostępności, relatywnie niskie koszty utrzymania danych oraz zoptymalizowany interfejs API dedykowany do bezpiecznego przesłania i pobierania dużych obiektów binarnych (ang. *Binary Large Objects*, BLOB).

3.5 Broker komunikatów — Azure Service Bus

Azure Service Bus [13] to chmurowa usługa kolejki komunikatów i zarządzania strumieniami danych klasy korporacyjnej. W architekturze systemu *AddiPi* pełni ona kluczową

rol brokera komunikatów (ang. *message broker*) odpowiedzialnego za asynchroniczne i bezprzerwowe pośrednictwo pomiędzy usługą *Files Service* (producentem komunikatów) a komponentem *Queue Service* (konsumentem) [14].

Dedykowana kolejka *print-queue* zapewnia niezawodną semantykę dostarczania wiadomości typu *at-least-once* (co najmniej raz) [3]. Rozwiązanie to gwarantuje, że w przypadku awarii usługi konsumenckiej lub utraty połączenia sieciowego, komunikat nie zostanie skasowany — pozostaje on zabezpieczony w strukturze magistrali chmurowej i może zostać przetworzony ponownie po przywróceniu operacyjności konsumenta, co bezpośrednio realizuje wymaganie niefunkcjonalne dotyczące tolerancji błędów.

3.6 Sterowanie urządzeniem — Azure IoT Hub

Azure IoT Hub [15] to usługa chmurowa przeznaczona do scentralizowanego zarządzania, bezpiecznego uwierzytelniania oraz monitorowania urządzeń Internetu Rzeczy (IoT). Usługa ta umożliwia pełną dwukierunkową komunikację (ang. *Device-to-Cloud* oraz *Cloud-to-Device*), a także udostępnia mechanizm metod bezpośrednich (ang. *Direct Methods*). Metody te stanowią synchroniczny wzorzec wywoła typu RPC (ang. *Remote Procedure Call*), realizowany na urządzeniach brzegowych komunikujących się za pośrednictwem protokołów MQTT lub AMQP.

W architekturze systemu *AddiPi* usługa *Printer Service* zdalnie wywołuje na agencie (uruchomionym na platformie Raspberry Pi) metody bezpośrednie: *startPrint*, *cancelPrint* oraz *getStatus*. W odpowiedzi zwrotnej agent transmituje asynchroniczny strumień danych telemetrycznych w postaci zdarzeń systemowych (m.in. *print_started*, *print_progress* czy *print_completed*). Komunikaty te są agregowane przez usługę *Printer Service* za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*.

3.7 Interfejs użytkownika — React oraz Vite

Warstwa prezentacji (interfejs użytkownika) została zaimplementowana z wykorzystaniem biblioteki *React* [16] w wersji 18 oraz języka TypeScript [6]. W charakterze środowiska deweloperskiego oraz narzędzia do automatyzacji procesu budowania (ang. *bundler*) aplikacji wykorzystano framework *Vite* [17], co pozwoliło na optymalizację czasu kompilacji kodu źródłowego.

Do zarządzania globalnym stanem aplikacji klienckiej zastosowano bibliotek *Zustand* [18]. Rozwiązanie to umożliwia wydajne współdzielenie danych pomiędzy komponentami drzewa DOM w sposób bezstanowy, eliminując konieczność implementowania nadmiarowej i zoonej architektury (takiej jak *Redux*) przy jednoczesnym zachowaniu wysokiej reaktywności interfejsu.

Projekt graficzny oraz warstwa stylów zostały zrealizowane przy użyciu frameworka *Tailwind CSS* [19]. Narzędzie to bazuje na architekturze klas narzędziowych (ang. *utility-first*), w której style interfejsu są komponowane bezpośrednio w strukturze kodu za pomocą predefiniowanych klas reprezentujących pojedyncze właściwości kaskadowych arkuszy stylów.

3.8 Konteneryzacja — Docker oraz Docker Compose

W celu zapewnienia powtarzalności środowiska uruchomieniowego, każdy z mikroservisów wchodzących w skład ekosystemu posiada dedykowany plik konfiguracyjny *Dockerfile*. Proces budowania obrazów realizuje mechanizm wieloetapowy (ang. *multi-stage build*). Podejście to pozwala na drastyczne ograniczenie rozmiaru finalnych obrazów kontenerów oraz całkowite odizolowanie środowiska kompilacji i zależności deweloperskich od produkcyjnego środowiska uruchomieniowego.

Narzędzie *Docker Compose* [20] wykorzystano do zautomatyzowanej orkiestracji i zarządzania cyklem życia wszystkich kontenerów zarówno w środowisku lokalnym, jak i produkcyjnym. Narzędzie to definiuje odizolowane, współdzielące wirtualne addipi-network oraz odpowiada za bezpieczne wstrzykiwanie zmiennych środowiskowych odczytywanych z pliku `.env`.

3.9 Automatyzacja infrastruktury chmurowej — Terraform

Do zarządzania rozproszonymi zasobami chmurowymi wykorzystano narzędzie *Terraform* [21], implementujące paradygmat deklaratywnego definiowania środowisk systemowych jako kod (ang. *Infrastructure as Code*, IaC). Architektura wdrożeniowa została podzielona na dwa niezależne moduły: `terraform/` dedykowany dla usług platformy *Microsoft Azure* (z użyciem dostawcy `hashicorp/azurerm`) oraz `terraform-oci/` konfiguruje zasoby w chmurze *Oracle Cloud Infrastructure (OCI)* [22] (oparty na dostawcy `oracle-oci`). Wybór tego rozwiązania gwarantuje pełną powtarzalność środowisk produkcyjnych oraz automatyzację procesów aprowizacji.

3.10 środowisko hostingowe warstwy serwerowej — Oracle Cloud Infrastructure

Kontenery aplikacyjne warstwy serwerowej (*Auth, User, Files, Queue* oraz *Printer Service*) są uruchamiane na maszynie wirtualnej wewnątrz chmury *Oracle Cloud Infrastructure* (OCI) [22]. W projekcie wykorzystano instancję VM.Standard.E2.1.Micro udostępnianą w bezpłatnym programie *Always Free* (specyfikacja sprzętowa: 1 OCPU, 1 GB pamięci RAM). Platformę OCI wybrano z uwagi na dostępność autonomicznych zasobów obliczeniowych działających bez restrykcji czasowych, co stanowi ekonomicznie optymalną alternatywę dla rozwiązań komercyjnych rozliczanych w modelu subskrypcyjnym lub za faktyczny czas pracy instancji.

3.11 CAD - SolidWorks

Projekt mechaniczny obudowy urządzenia wykonano w programie **SolidWorks** [23], stanowiącym środowisko do parametrycznego modelowania bryowego oraz projektowania złożeń mechanicznych. Oprogramowanie umożliwia tworzenie asocjatywnych modeli 3D, w których modyfikacja parametrów części powoduje automatyczną aktualizację złożeń oraz dokumentacji technicznej.

Wybór programu SolidWorks był podyktowany dostępnością oprogramowania w ramach licencji studenckiej oraz szerokim wsparciem dla formatów eksportu modeli, takich jak .STEP (neutralny format wymiany modeli CAD), .STL (format wykorzystywany w procesie przygotowania modeli o druku 3D) oraz .3MF [1] (format wymiany zawierający dodatkowe metadane procesu druku).

Projekt został podzielony na pliki .SLDPRT, reprezentujące pojedyncze części, oraz .SLDASM, definiujące kompletne złozenia mechaniczne.

3.12 Porównanie wybranych alternatyw

Proces projektowania systemu wymaga wielokryterialnej analizy dostępnych rozwiązań technologicznych. Decyzje architektoniczne oparto na kryteriach takich jak: stopień integracji z chmurą obliczeniową, łatwość skalowania, nakłady na zarządzanie infrastrukturą oraz spójność ekosystemu programistycznego. Zbiorcze zestawienie wszystkich podjętych decyzji projektowych, wybranych technologii oraz odrzuconych alternatyw przedstawiono na

rysunku 1.

Kategoria	Wybrana technologia	Odrzucone alternatywy	Kluczowy powód
Język backendowy serwisy Node.js + agent	TypeScript / Node.js + Python (agent RPi) Express.js v5	Go, Java, PHP	statyczne typowanie, duży ekosystem npm, nieblokujące I/O
Baza danych users, jobs, tokens	Azure Cosmos DB SQL API (JSON) PaaS, bez serwera	PostgreSQL, MongoDB Atlas	natywna integracja z Azure, elastyczny schemat JSON
Kolejka komunikatów upload → zadanie	Azure Service Bus kolejka print-queue at-least-once, PaaS	RabbitMQ, Apache Kafka	brak zarządzania serwerem, auto- skalowanie
Sterowanie IoT chmura → Raspberry Pi	Azure IoT Hub Direct Methods, MQTT telemetry D2C	AWS IoT Core, MQTT broker własny	Direct Methods, ekosystem Azure, telemetry wbudowana
Frontend aplikacja webowa	React 18 + Vite TypeScript, Zustand Tailwind CSS, i18next	Vue, Angular, Next.js	dojrzały ekosystem, hooks, SPA bez SSR (prostota)
Wdrożenie konteneryzacja	Docker + Compose multi-stage build addip-network	Kubernetes, Podman, bare metal	przenośność, jedna komenda uruchomienia, izolacja serwisów

Rysunek 1: Zbiorcze zestawienie wybranej architektury technologicznej, odrzuconych alternatyw oraz kluczowych powodów decyzyjnych.

Głównym zaoeniem przy wyborze warstwy backendowej oraz frontendowej bylo ujedno-liczenie stosu technologicznego wokó języka TypeScript oraz rodowiska Node.js. Pozwolilo to na współdzielenie typów danych midzy aplikacj webow (React 18 + Vite) a usugami serwerowymi, co znaczo przyspieszylo proces implementacji. Wyjtek stanowi mikrousuga *Files Service* oraz komponent agenta uruchamiany na platformie Raspberry Pi, gdzie ze wzgldu na specyfik obsugi interfejsów sprztowych zdecydowano si na wykorzystanie języka Python [9].

Kluczowe znaczenie dla stabilnoci systemu miao precyzyjne dobranie silnika bazy danych oraz mechanizmu wymiany komunikatów. Szczegóowe porównanie cech rozwa-anych systemów bazodanowych oraz systemów kolejkowych zestawiono odpowiednio w tabeli 1 oraz tabeli 2.

Tabela 1: Porównanie baz danych rozważanych w projekcie

Cecha	Cosmos DB [11]	PostgreSQL [24]	MongoDB Atlas [25]
Model danych	Dokument (JSON)	Relacyjny	Dokument (BSON)
Zarządzanie	PaaS (bez serwera)	Samodzielny/PaaS	PaaS
Integracja z Azure	Natywna	Ograniczona	Poreдна
Elastyczny schemat danych	Tak	Nie	Tak
Dostępno bezpłatnego planu	Tak (400 RU/s)	Tak (self-hosted)	Tak (512 MB)

Jak wynika z tabeli 1, baza *Azure Cosmos DB* została wybrana ze względu na bezobsługowy model bezserwerowy (Serverless PaaS) oraz natywną integrację z ekosystemem *Azure*. Wybór modelu NoSQL zamiast relacyjnego PostgreSQL [24] podyktowany był dążeniem do minimalizacji narzutów związanych z migracjami schematu, co w systemach intensywnie korzystających z danych pozwala na szybszą iterację i skalowanie [3].

Tabela 2: Porównanie rozwiązań kolejkowania wiadomości

Cecha	Azure Service Bus	RabbitMQ [26]	Apache Kafka [27]
Model wdrożenia	PaaS	Self-hosted	Self-hosted / PaaS
Semantyka dostarczania	At-least-once	At-least-once	At-least-once
Wzrost konfiguracji	Niska	średnia	Wysoka
Integracja z Azure	Natywna	Ograniczona	Poreдна
Skalowalność	Automatyczna	Ręczna	Wysoka

W przypadku infrastruktury kolejki (tab. 2), usługa *Azure Service Bus* [13] przewyższa alternatywy brakiem konieczności samodzielnego zarządzania serwerem oraz wbudowaną automatyczną skalowalnością. Narzut konfiguracyjny rozwiązań takich jak *Apache Kafka* [27] okazał się nieuzasadniony w stosunku do skali projektu, natomiast *Azure Service Bus* zapewni wymagania semantyki dostarczania wiadomości *at-least-once* bezpośrednio po uruchomieniu usługi.

Ostatnim etapem konfiguracji było zdefiniowanie środowiska uruchomieniowego. Odrzucono zoone systemy orkiestracji (Kubernetes) oraz instalacje bezpośrednie (bare metal) na rzecz narzędzia Docker + Compose. Zapewniono to przez przeniesienie kodu, powtarzalność wdrożenia przy użyciu wieloetapowego budowania obrazów (*multi-stage build*) oraz bezpieczną izolację usług w ramach dedykowanej sieci wirtualnej o nazwie *addipi-network*.

4 Architektura systemu

4.1 Ogólna struktura

System *AddiPi* zbudowany jest w architekturze mikroserwisowej [14] [28]. Każdy serwis stanowi niezależną jednostkę wdrożeń odpowiedzialną za określony domenę biznesową oraz komunikuje się z pozostałymi komponentami za pośrednictwem zdefiniowanych interfejsów (HTTP REST lub kolejka komunikatów).

W ramach systemu wyodrębniono następujące komponenty:

1. **AddiPi Auth Service** (port 3001) - realizuje proces uwierzytelniania użytkowników oraz obsługę tokenów JWT.
2. **AddiPi User Service** (port 3002) - odpowiada za zarządzanie profilami użytkowników oraz historią zadań do wydruku.
3. **AddiPi Files Service** (port 5000) - umożliwia przesyłanie oraz przechowywanie plików G-code.
4. **AddiPi Queue Service** (porty 3070) - odbiera komunikaty z kolejki oraz zapisuje informacje o zadaniach w bazie *Azure Cosmos DB*.
5. **AddiPi Printer Service** (port 3050) - odpowiada za harmonogramowanie zadań do wydruku, komunikację z *Azure IoT Hub* [15] oraz odbiór telemetrii urządzenia.
6. **AddiPi Video Service** (port 3003) - odpowiada za rejestrację i udostępnianie adresu URL kamery systemu *OctoPrint*.
7. **AddiPi Agent** (Raspberry Pi) - agent uruchamiany na urządzeniu Raspberry Pi, pośredniczy pomiędzy *Azure IoT Hub* a systemem *OctoPrint*.
8. **AddiPi Frontend** (Vercel [29] / port 5173) - aplikacja webowa zbudowana z wykorzystaniem biblioteki *React* [16] oraz narzędzia *Vite* [17].

4.2 Diagram architektury

Na rysunku 2 przedstawiono sekwencyjny przepływ danych pomiędzy poszczególnymi komponentami zaprojektowanego systemu.

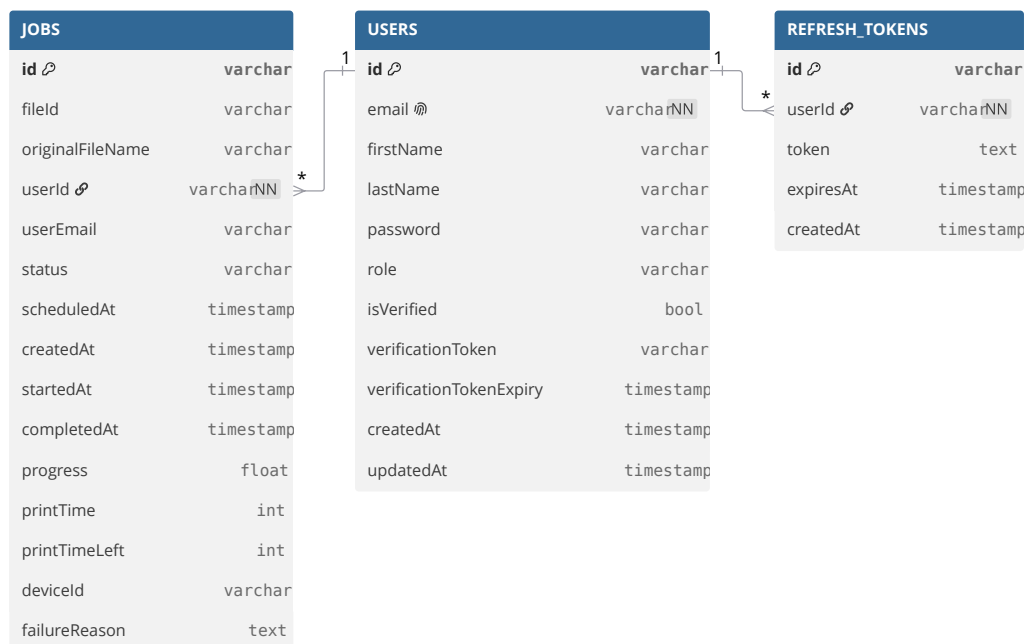
kujcych na realizacj. Jeeli urzdzienie drukujce jest wolne, a zadanie spenia kryteria rozpoczcia procesu wytwórczego, status transakcji jest zmieniany na `waiting_for_printer_ready`. System przechodzi wówczas w stan oczekiwania na potwierdzenie gotowoci operacyjnej drukarki.

5. **Inicjacja komunikacji IoT:** Po autoryzacji gotowoci urzdzienia przez uytownika lub administratora, usuga *Printer Service* zdalnie wywouje metod bezporedni (ang. *Direct Method*) `startPrint` na dedykowanym agencie, wykorzystujc do tego celu usug *Azure IoT Hub*.
6. **Zarzdanie procesem drukowania przez agenta:** Komponent *AddiPi Agent* pobiera przypisany plik G-code z repozytorium *Azure Blob Storage* [12], przekazuje go bezporednio do interfejsu oprogramowania *OctoPrint* oraz inicjuje proces drukowania 3D. W trakcie pracy agent w sposób cigy transmituje dane telemetryczne do usugi *Azure IoT Hub*, raportujc biecy postp, status ukoczenia lub ewentualne bdy krytyczne.
7. **Odbiór telemetry i aktualizacja stanu:** Usuga *Printer Service* konsumuje strumie danych telemetrycznych za porednictwem punktu kocowego zgodnego ze specyfikacj *Azure Event Hubs*, a nastpnie aktualizuje stan procesu w bazie *Azure Cosmos DB*.
8. **Odwanie danych w interfejsie:** Aplikacja kliencka (frontendowa) realizuje cykliczne zapytania sondujce (ang. *polling* z interwaem 5 sekund) do mikroserwisów, zapewniajc bie aktualizacj stanu systemu prezentowanego uytownikowi.

4.3 Model danych

W bazie danych *Azure Cosmos DB* [11] dane s strukturyzowane i przechowywane w postaci dokumentów w formacie JSON [3]. Zostay one pogrupowane w logicznych kontenerach, odpowiadajcych poszczególnym domenom biznesowym systemu.

Na rysunku 3 przedstawiono fizyczny diagram encji i relacji (ang. *Entity-Relationship Diagram*, ERD) zaprojektowanej bazy danych *addipi-cosmos*. Diagram ten odwzorowuje logiczne powizania referencyjne pomidzy dokumentami w poszczególnych kontenerach.



Rysunek 3: Diagram bazy danych addipi-cosmos w notacji Crow's Foot (ptasiej apki).

4.3.1 Użytkownik (kontener users)

Dokument przechowuje informacje profilowe użytkowników, dane autoryzacyjne oraz status weryfikacji konta. Przykładowy struktur przedstawiono na listingu 1. Pole password przechowuje bezpieczny skrót hasa wygenerowany przy użyciu opisanego dalej algorytmu *bcrypt* [4].

```

1  {
2    "id": "user_<timestamp>",
3    "email": "student@uwr.edu.pl",
4    "firstName": "Jan",
5    "lastName": "Kowalski",
6    "password": "<bcrypt hash>",
7    "role": "user",
8    "isVerified": true,
9    "verificationToken": "<hex>",
10   "verificationTokenExpiry": "2025-01-01T00:00:00Z",
11   "createdAt": "2025-01-01T00:00:00Z",
12   "updatedAt": "2025-01-01T00:00:00Z"
13 }
  
```

Listing 1: Przykładowy dokument użytkownika w Azure Cosmos DB

Pole role definiuje uprawnienia w systemie i może przyjmować wartość user (użytkownik

standardowy) lub admin (administrator systemu).

4.3.2 Zadanie do wydruku (kontener jobs)

Dokument zadania (listing 2) agreguje informacje o pliku sterującym, powizanym użytkownikowi, metrykach czasu oraz bieżącym stanie realizacji procesu produkcyjnego.

```
1  {
2    "id": "<timestamp string>",
3    "fileId": "20251203_014648_model.gcode",
4    "originalFileName": "model.gcode",
5    "userId": "user_<timestamp>",
6    "userEmail": "student@uwr.edu.pl",
7    "status": "printing",
8    "scheduledAt": "2025-12-01T15:00:00Z",
9    "createdAt": "2025-11-30T10:00:00Z",
10   "startedAt": "2025-12-01T15:01:00Z",
11   "completedAt": "2025-12-01T17:30:00Z",
12   "progress": 87.5,
13   "printTime": 4500,
14   "printTimeLeft": 600,
15   "deviceId": "raspberry-pi-mkt-01",
16   "failureReason": null
17 }
```

Listing 2: Przykładowy dokument zadania do wydruku w Cosmos DB

Pole status może przyjmować wartości: pending, scheduled, waiting_for_printer_ready, delayed, printing, completed, failed, cancelled.

4.3.3 Token odwołania (kontener refresh-tokens)

W celu zapewnienia bezpiecznej, bezstanowej autoryzacji sesji, tokeny odwołania (ang. *refresh tokens*) są utrwalane w dedykowanym kontenerze (listing 3).

```
1  {
2    "id": "<userId>_<timestamp>",
3    "userId": "user_<timestamp>",
4    "token": "<JWT string>",
5    "expiresAt": "2025-12-08T00:00:00Z",
6    "createdAt": "2025-12-01T00:00:00Z"
```

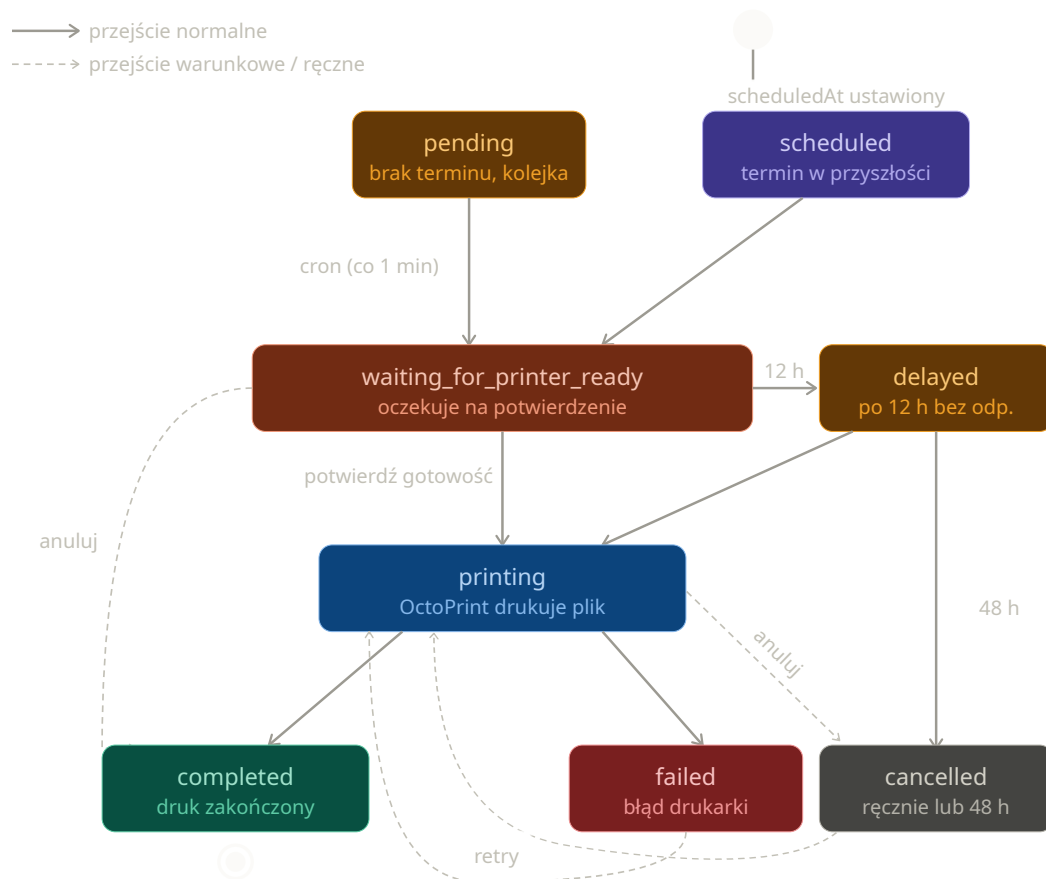

Listing 3: Przykładowy dokument refresh tokena

4.4 Cykl życia zadania do wydruku

Zadanie w procesie przetwarzania i realizacji może znajdować się w jednym z następujących stanów:

- **pending** — zadanie bez określonego terminu realizacji, oczekujące w kolejce ogólnej systemu.
- **scheduled** — zadanie posiadające ściśle zaplanowany termin realizacji, oczekujące na osiągnięcie wskazanego czasu rozpoczęcia. Po nadejściu zaplanowanego czasu zadanie to otrzymuje najwyższy priorytet wykonania.
- **waiting_for_printer_ready** — usługa harmonogramująca (ang. *scheduler*) wybiera zadanie do realizacji; system oczekuje na manualne potwierdzenie gotowości urządzenia przez użytkownika.
- **delayed** — stan opóźnienia, w którym system oczekuje na potwierdzenie gotowości drukarki dłużej niż 3 h; do użytkownika wysyłane jest automatyczne powiadomienie przypominające.
- **printing** — proces drukowania modelu sterującego jest aktywnie realizowany przez oprogramowanie *OctoPrint*.
- **completed** — proces drukowania 3D został pomyślnie sfinalizowany.
- **failed** — w trakcie realizacji procesu produkcyjnego wystąpił błąd krytyczny uniemożliwiający dalsze drukowanie.
- **cancelled** — zadanie zostało anulowane ręcznie przez użytkownika lub administratora lub automatycznie przez system po upływie 48 h oczekiwania na potwierdzenie gotowości urządzenia.

Na rysunku 4 przedstawiono formalny model maszyny stanów, obrazujący dopuszczalne przejścia fazowe w cyklu życia zadania produkcyjnego.



Rysunek 4: Diagram maszyny stanów cyklu życia zadania do wydruku.

4.5 Bezpieczeństwo i uwierzytelnianie

W celu zapewnienia poufności danych oraz kontroli dostępu, w systemie zaimplementowano dwutokenowy model uwierzytelniania oparty na standardzie JWT (*JSON Web Token*) [2] [28]. Mechanizm ten wykorzystuje dwa rodzaje obiektów:

- **Access token** — token dostępu o krótkim czasie autoryzacji (wany przez 15 minut), przekazywany w nagłówku protokołu HTTP (`Authorization: Bearer <token>`). Integralność i autentyczność tokenu są weryfikowane kryptograficznie przy użyciu klucza symetrycznego `JWT_SECRET`.
- **Refresh token** — token odwołujący o wydłużonym czasie ważności (wynoszącym 7 dni), utrwalany w bazie danych *Azure Cosmos DB* oraz przechowywany po stronie aplikacji klienckiej. Podpisywany jest przy użyciu dedykowanego klucza `JWT_REFRESH_SECRET`. Służy do generowania nowej pary tokenów bez konieczności ponownego wprowadzania danych uwierzytelniających przez użytkownika.

Hasła użytkowników nie są przechowywane w bazie danych w postaci jawnej. Do ich zabezpieczenia wykorzystano algorytm funkcyjny skrótu *bcrypt* [4] z parametrem kosztu (ang. *salt rounds*) ustawionym na wartość 10.

Proces rejestracji nowych kont został ze względów bezpieczeństwa ograniczony wyłącznie do adresów poczty elektronicznej w domenie akademickiej @uwr.edu.pl. Po aktywacji profilu wymaga weryfikacji dwuetapowej poprzez kliknięcie w unikalny link aktywacyjny przesłany na adres e-mail użytkownika.

Kontrola dostępu na poziomie kodu realizowana jest za pomocą oprogramowania pośredniczego (ang. *middleware*):

- `requireAuth` — odpowiada za ekstrakcję i weryfikację tokenu dostępu poprzez wysłanie wewnętrznego zapytania weryfikacyjnego do punktu końcowego `POST /auth/verify` usługi *Auth Service*.
- `requireAdmin` — realizuje mechanizm kontroli dostępu opartej na rolach (ang. *Role-Based Access Control*, RBAC), sprawdzając, czy zdekodowany profil użytkownika posiada przypisane uprawnienia klasy `admin`.

Takie podejście pozwala na odseparowanie logiki biznesowej od mechanizmów infrastrukturalnych, co zwiększa czytelność i ułatwia testowanie poszczególnych modułów systemu [7].

5 Implementacja serwisów warstwy serwerowej

5.1 Auth Service

5.1.1 Opis ogólny

Usługa *Auth Service* odpowiada za procesy uwierzytelniania, autoryzacji oraz zarządzania kontami użytkowników. W zakres jej funkcjonalności wchodzi: rejestracja, logowanie i wylogowanie użytkowników [28], odwołanie tokenów sesyjnych oraz weryfikacja adresów poczty elektronicznej.

Komponent ten został zaimplementowany w środowisku uruchomieniowym Node.js [8] z wykorzystaniem frameworka Express.js [10] oraz języka TypeScript [6]. Serwis realizuje operacje wejścia-wyjścia poprzez komunikację z bazą danych *Azure Cosmos DB* (w ramach kontenerów *users* oraz *refresh-tokens*). Ponadto integruje się z zewnętrznym serwerem SMTP firmy Google w celu dystrybucji wiadomości aktywacyjnych, wykorzystując do tego celu bibliotekę *Nodemailer* [30].

5.1.2 Punkty końcowe API

W tabeli 3 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez usługę *Auth Service*.

Tabela 3: Punkty końcowe usługi *Auth Service*.

Metoda	ścieżka	Opis
POST	/auth/register	Rejestracja nowego użytkownika, walidacja domeny e-mail, generowanie szyfru hasła oraz wysłanie wiadomości z linkiem aktywacyjnym.
POST	/auth/login	Uwierzytelnienie użytkownika; generowanie i zwrócenie pary tokenów: <i>access token</i> oraz <i>refresh token</i> .
POST	/auth/logout	Unieważnienie tokenu odwołującego i zakończenie aktywnej sesji użytkownika.

Tabela 3 – Kontynuacja z poprzedniej strony

Metoda	cieka	Opis
PATCH	/auth/refresh	Generowanie nowego tokenu dostpu na podstawie wanego tokenu odwieajcego.
POST	/auth/verify	Weryfikacja poprawnoci tokenu dostpu, wykorzystywana wewnatrznie przez pozostae mikroserwisy.
GET	/auth/verify-email	Aktywacja konta uytownika po odebraniu i weryfikacji tokenu z linku aktywacyjnego.
POST	/auth/resend-verification	Ponowne wygenerowanie i wysanie wiadomoci weryfikacyjnej na adres e-mail.
GET	/health	Punkt kocowy diagnostyki stanu dziaania usugi (ang. <i>health check</i>).

5.1.3 Kluczowe fragmenty implementacji

Na listingu 4 przedstawiono implementacj funkcji odpowiedzialnych za generowanie tokenu dostpu oraz tokenu odwieajcego przy uyciu biblioteki jsonwebtoken [2].

```

1 export function generateAccessToken(user: User): string {
2   const payload: JWTPayload = {
3     userId: user.id,
4     email: user.email,
5     role: user.role
6   };
7   return jwt.sign(payload, CONFIG.JWT_SECRET, { expiresIn: '15m'
8     });
9 }
10 export function generateRefreshToken(user: User): string {
11   return jwt.sign(
12     { userId: user.id },
13     CONFIG.JWT_REFRESH_SECRET,

```

```

14 { expiresIn: '7d' }
15 );
16 }

```

Listing 4: Implementacja generowania tokenów JWT (token-service.ts)

Proces walidacji tokenu odwieajcego (ang. *refresh token*) jest dwuetapowy. Obejmuje zarówno kryptograficzn weryfikacj poprawnoci podpisu cyfrowego struktury JWT, jak i asynchroniczne sprawdzenie obecności oraz wanoci powizanego rekordu w bazie danych *Azure Cosmos DB*. Takie podejcie architektoniczne umoliwia natychmiastowe, zdalne uniewnianie sesji (ang. *token revocation*) w momencie wylogowania si uytownika z systemu. Kod realizujcy t procedur zaprezentowano na listingu 5.

```

1 export async function validateRefreshToken(
2   token: string
3 ): Promise<string | null> {
4   try {
5     const decoded = jwt.verify(
6       token,
7       CONFIG.JWT_REFRESH_SECRET
8     ) as { userId: string };
9
10    const query = `SELECT * FROM c
11      WHERE c.token = @token
12      AND c.userId = @userId
13      AND c.expiresAt > @now`;
14
15    const { resources } = await refreshTokensContainer.items.query(
16      {
17        query,
18        parameters: [
19          { name: '@token', value: token },
20          { name: '@userId', value: decoded.userId },
21          { name: '@now', value: new Date().toISOString() }
22        ]
23      }).fetchAll();
24
25    return resources.length > 0 ? decoded.userId : null;
26  } catch {
27    return null;
28  }
29 }

```

```
27 }  
28 }
```

Listing 5: Implementacja walidacji refresh tokenu (`token-service.ts`)

5.1.4 Weryfikacja adresu poczty elektronicznej

Po pomylnym zakoczeniu procesu rejestracji system generuje unikalny, 64-znakowy token w formacie szesnastkowym. Jest on tworzony przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych z modułu `crypto` (`crypto.randomBytes`). Zapewnia to wysoką entropię i odporność na ataki typu *brute-force*.

Wygenerowany identyfikator jest zapisywany w bazie danych wraz z datą wygaśnięcia, a do użytkownika wysyłana jest wiadomość e-mail zawierająca odnośnik aktywacyjny skierowany do punktu końcowego systemu:

```
GET /auth/verify-email?token=<hex>
```

Po odebraniu danych przez usługę *Auth Service*, system przeprowadza weryfikację poprawności oraz wygaśnięcia tokenu. W przypadku pozytywnego rezultatu walidacji, atrybut `isVerified` w dokumencie użytkownika przyjmuje wartość `true`, a konto zostaje w pełni aktywowane. Na zakończenie procesu serwer wykonuje przekierowanie (ang. *HTTP redirect*) przeglądarki użytkownika do adresu bazowego aplikacji klienckiej.

5.2 User Service

5.2.1 Opis ogólny

Usługa *User Service* realizuje zadania związane z zarządzaniem profilami użytkowników, modyfikacją ich uprawnień oraz agregacją danych dotyczących powiązanych zadań wytwórczych. Komponent ten wykonuje operacje odczytu i zapisu danych w ramach kontenerów `users` oraz `jobs` w bazie danych *Azure Cosmos DB*.

Autoryzacja oraz weryfikacja tożsamości użytkowników są realizowane w sposób scentralizowany poprzez przesłanie wewnętrznych zapytań do punktu końcowego `POST /auth/verify` opisanej wcześniej usługi *Auth Service*.

5.2.2 Punkty kocowe API

W tabeli 4 wyszczególniono i opisano punkty kocowe (ang. *endpoints*) udostępniane przez usług *User Service*, z uwzględnieniem podziału na operacje standardowe oraz administracyjne.

Tabela 4: Punkty kocowe usługi User Service.

Metoda	ścieżka	Opis
GET	/users/me	Pobranie danych profilowych zalogowanego użytkownika.
PATCH	/users/me	Aktualizacja danych osobowych użytkownika (imię oraz nazwisko).
GET	/users/me/jobs	Pobranie historii wszystkich zadań przypisanych do zalogowanego użytkownika.
GET	/users/me/stats	Pobranie statystyk użytkownika, obejmujących m.in. zagregowane liczby zadań z podziałem na ich statusy.
GET	/users/jobs/upcoming	Pobranie listy zadań oczekujących na realizację (posiadających status pending lub scheduled).
GET	/users/jobs/recent-completed	Pobranie historycznej listy ostatnio sfinalizowanych procesów druku.
GET	/users	Pobranie rejestru wszystkich użytkowników systemu (dostęp zastrzeżony dla administratora).
PATCH	/users/:id/role	Modyfikacja roli i poziomu uprawnień użytkownika o wskazanym identyfikatorze (dostęp zastrzeżony dla administratora).

Tabela 4 – ciąg dalszy z poprzedniej strony

Metoda	cieka	Opis
DELETE	/users/:id	Trwa usunięcie konta użytkownika na podstawie parametru identyfikatora (dostęp zastrzeżony dla administratora).
GET	/users/:id/jobs	Pobranie pełnej listy zadań powiązanych z wybranym kontem użytkownika (dostęp zastrzeżony dla administratora).

5.2.3 Mechanizm autoryzacji

Na listingu 6 zaprezentowano produkcyjną implementację oprogramowania pośredniczącego `requireAuth`. Komponent ten w pełni wykorzystuje silne typowanie języka TypeScript, w tym parametry generyczne interfejsów `Request` oraz `Response` z biblioteki `express` [10].

Procedura autoryzacji polega na bezpiecznej ekstrakcji tokenu dostępu z nagłówka `Authorization`, a następnie wykonaniu asynchronicznego dania sieciowego metod `POST` do punktu końcowego usługi *Auth Service*. Po odebraniu odpowiedzi struktura danych jest rzutowana na dedykowany typ `Data`. W przypadku poprawnej walidacji następuje asercja typu obiektu dania, wstrzyknięcie profilu użytkownika do pola `user` oraz wywołanie funkcji `next()`, która przekazuje sterowanie do kolejnego elementu w potoku przetwarzania [7] (ang. *middleware chain*).

```

1 import { CONFIG } from "../config/config";
2 import type { Request, Response, NextFunction } from "express";
3 import { Data, AuthUser } from "../mwTypes";
4
5 const requireAuth = async (
6   req: Request<{}>, unknown, {}, {}>,
7   res: Response<{error: string}>,
8   next: NextFunction
9 ): Promise<void | {error: string}> => {

```

```

10 try {
11     const token: string | undefined = req.headers.authorization?.
        replace('Bearer ', '');
12
13     if (!token) {
14         res.status(401).json({ error: 'Missing authentication token'
        });
15         return;
16     }
17
18     const response: globalThis.Response = await fetch(`${CONFIG.A
        UTH_SERVICE_URL}/auth/verify`, {
19         method: 'POST',
20         headers: {
21             'Authorization': `Bearer ${token}`,
22             'Content-Type': 'application/json'
23         }
24     });
25
26     const data: Data = await response.json() as Data;
27
28     if (!response.ok) {
29         throw new Error('Authentication failed');
30     }
31
32     (req as any).user = data.user;
33
34     next();
35 } catch (err) {
36     res.status(401).json({ error: 'Invalid authentication token'
        });
37 }
38 };
39
40 export default requireAuth;

```

Listing 6: Implementacja oprogramowania pośredniczego requireAuth (requireAuth.ts)

5.3 Files Service

5.3.1 Opis ogólny

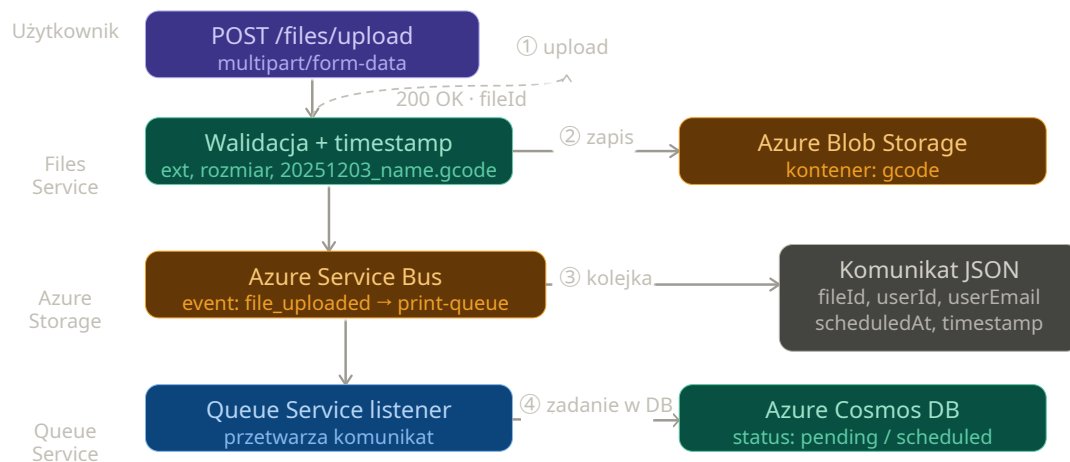
Usługa *Files Service* stanowi mikroserwis zaimplementowany w języku Python [9] z wykorzystaniem mikroframeworka Flask [31]. Komponent ten odpowiada za asynchroniczne przyjmowanie plików z kodem sterującym (G-code) od aplikacji klienckiej, ich bezpieczne utrwalanie w repozytorium obiektowym *Azure Blob Storage* [12] oraz rozgaszanie zdarzeń systemowych za pośrednictwem magistrali komunikatów *Azure Service Bus* [13].

5.3.2 Procedura przetwarzania i przesyłania plików

Przebieg procesu przesyłania pliku oraz jego rejestracji w systemie realizowany jest sekwencyjnie według następującego scenariusza:

1. **Inicjacja dania:** Aplikacja kliencka wysyła dane POST `/files/upload` zawierające plik zakodowany w formacie `multipart/form-data`.
2. **Weryfikacja uprawnień:** Oprogramowanie pośredniczące `require_auth` deleguje proces walidacji tokenu uwierzytelniającego do usługi *Auth Service*.
3. **Walidacja potoku wejściowego:** System weryfikuje poprawno formatu pliku (akceptowane jest wyłącznie rozszerzenie `.gcode`), jego rozmiar (wprowadzono limit do 50 MB) oraz opcjonalnie strukturę wewnętrzną (w przypadku aktywacji flagi konfiguracyjnej `STRICT_CONTENT_CHECK`).
4. **Sanitaryzacja i unikalność nazw:** Nazwa pliku wejściowego jest modyfikowana poprzez dodanie prefiksu czasowego zawierającego kompletną datę i godzin odebrania dania (np. `20251203_014648_model.gcode`), co zapobiega konfliktom nadpisywania zasobów.
5. **Utrwalenie zasobu:** Plik w formacie binarnym zostaje przesłany i zapisany w dedykowanym kontenerze `gcode` w ramach usługi *Azure Blob Storage*.
6. **Publikacja zdarzenia:** Do kolejki `print-queue` magistrali *Azure Service Bus* wysyłany jest komunikat w formacie JSON, który inicjuje dalsze etapy cyklu życia zadania.

Na rysunku 5 zobrazowano schematyczny przepływ danych podczas procedury przesyłania i kolejkowania pliku G-code.



Rysunek 5: Przepływ przesyłania pliku G-code — od zapytania HTTP do integracji z kolejką.

Na listingu 7 zaprezentowano implementację końcowego etapu obsługi dania w ramach kontrolera usługi *Files Service*. Po pomyślnej walidacji i zapisie pliku w repozytorium obiektowym, struktura danych reprezentująca zdarzenie jest serializowana do formatu JSON i przekazywana do metody nadawczej klienta *Azure Service Bus*. Cała operacja została otoczona blokiem obsługi wyjątków (`try-except`), co gwarantuje stabilność działania mikroservisu i zwrócenie ustrukturyzowanej odpowiedzi o b.dzie w formacie JSON wraz ze statusem HTTP 500 w przypadku awarii infrastruktury chmurowej.

```
1 message = {
2     'event': 'file_uploaded',
3     'fileId': filename,
4     'originalFileName': original_filename,
5     'userId': user_id,
6     'userEmail': user_email,
7     'timestamp': str(time.time()),
8     'scheduledAt': request.form.get('scheduledAt')
9 }
10
11 sb_client = current_app.config.get('SERVICE_BUS_CLIENT')
```

```

12     if sb_client:
13         with sb_client:
14             sender = sb_client.get_queue_sender(queue_name='print-
queue')
15             sender.send_messages([{'body': json.dumps(message)}])
16
17     return jsonify({'status': 'success', 'fileId': filename}), 200
18
19 except Exception as e:
20     return jsonify({'error': str(e)}), 500

```

Listing 7: Implementacja publikacji komunikatu w Azure Service Bus (files_controller.py)

5.4 Queue Service

5.4.1 Opis ogólny

Usługa *Queue Service* to mikroservis zaimplementowany w środowisku uruchomieniowym Node.js [8] przy użyciu języka JavaScript oraz standardu modułów ECMAScript (ang. *ECMAScript Modules*, ESM). Głównym zadaniem komponentu jest asynchroniczne i bezprzerwowe nasuchiwanie (ang. *listening*) komunikatów publikowanych w magistrali *Azure Service Bus* [14], a następnie ich transformacja i utrwalanie w postaci dokumentów zada produkcyjnych w bazie danych *Azure Cosmos DB*.

Dodatkowo serwis udostępnia dedykowany interfejs programistyczny HTTP API, który umożliwia innym komponentom systemu (w tym aplikacji klienckiej) monitorowanie stanu kolejki oraz zarządzanie priorytetami zada.

5.4.2 Mechanizm nasuchiwania kolejki

Na listingu 8 przedstawiono implementację procedury odpowiedzialnej za rejestrację i obsługę potoku zdarzeń pochodzących z kolejki chmurowej. Logika biznesowa realizuje bezpieczne parsowanie komunikatów w formacie JSON oraz obsługuje scenariusz awaryjny (ang. *dead-lettering/abandonment*) [14] poprzez wywołanie metody `abandonMessage()`, co pozwala na zachowanie spójności danych w przypadku wystąpienia błędów infrastrukturalnych.

```

1 export function startPrintQueueListener(container, receiver) {

```

```

2   if (!container) throw new Error('container is required for
    printQueueListener');
3   if (!receiver) throw new Error('receiver is required for
    printQueueListener');
4
5   const messageHandler = async (message) => {
6     try {
7       let data = message.body;
8       if (typeof data === 'string') {
9         try {
10          data = JSON.parse(data);
11        } catch(e) {
12          await receiver.abandonMessage(message);
13          return;
14        }
15      }
16
17      if (!data || data.event !== 'file_uploaded') {
18        return;
19      }
20
21      const job = {
22        id: Date.now().toString(),
23        fileId: data.fileId,
24        originalFileName: data.originalFileName || null,
25        userId: data.userId,
26        userEmail: data.userEmail,
27        status: data.scheduledAt ? 'scheduled' : 'pending',
28        scheduledAt: data.scheduledAt || null,
29        createdAt: getLocalISO(),
30      };
31
32      try {
33        await container.items.upsert(job);
34      } catch(upErr) {
35        throw upErr;
36      }
37    } catch(err) {
38      try {

```

```

39     await receiver.abandonMessage(message);
40   } catch(e) {
41     // Obsługa błąd komunikacji z repozytorium kolejki
42   }
43 }
44 };
45
46 const errorHandler = (error) => {
47   console.error('EVENT ERROR:', error);
48 };
49
50 receiver.subscribe({
51   processMessage: messageHandler,
52   processError: errorHandler
53 });
54 }

```

Listing 8: Implementacja mechanizmu nasuchiwanie kolejki (printQueueListener.js)

5.4.3 Punkty koczowe HTTP API

W tabeli 5 zestawiono punkty koczowe udostpniane przez interfejs sieciowy usugi *Queue Service*.

Tabela 5: Punkty koczowe usugi Queue Service.

Metoda	cieka	Opis
GET	/queue	Pobranie spaginowanej listy zada z moliwoci definiowania filtrów wyszukiwania.
GET	/queue/next	Pobranie kolejnego zadania przeznaczonego bezporednio do realizacji (zwraca kod 204 No Content w przypadku braku pozycji).
PATCH	/queue/cancel/:id	Anulowanie wybranego zadania drukowania (dostp ograniczony do roli administratora).

Tabela 5 – ciąg dalszy z poprzedniej strony

Metoda	cieka	Opis
GET	/queues	Pobranie metryk i informacji o stanie kolejek w ramach usługi <i>Azure Service Bus</i> (dostęp dla administratora).
GET	/health	Punkt kocowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

5.5 Printer Service

5.5.1 Opis ogólny

Usługa *Printer Service* stanowi centralny komponent orkiestracji systemu, odpowiedzialny za zaawansowane harmonogramowanie, weryfikację stanów urządzeń oraz nadzór nad realizacją procesów wytwórczych. Mikroserwis ten został zaimplementowany w języku TypeScript [6] i składa się z trzech autonomicznych podsystemów logicznych:

- **Harmonogram zadań** (uruchamiany cyklicznie z interwałem jednorazowym) — odpowiada za sekwencyjny wybór kolejnego zadania produkcyjnego przeznaczonego do realizacji na podstawie kryteriów czasowych.
- **Monitor gotowości urządzeń** (uruchamiany cyklicznie z interwałem pięciominutowym) — nadzoruje i weryfikuje statusy zadań oczekujących na manualne potwierdzenie gotowości drukarki, egzekwując zdefiniowane limity czasowe (ang. *timeouts*).
- **Odbiornik telemetrii** (ang. *Telemetry Listener*) — asynchroniczny komponent zintegrowany z usługą *Azure Event Hubs*, którego zadaniem jest ciągły odbiór zdarzeń telemetrycznych transmitowanych przez agenta sprzętowego oraz bieżąca aktualizacja stanów zadań w bazie danych.

5.5.2 Mechanizm harmonogramowania

W ramach każdej minutowej iteracji usługa harmonogramująca wykonuje sekwencję operacji kontrolno-decyzyjnych:

1. **Weryfikacja aktywnego procesu:** System sprawdza, czy w bazie danych istnieje zadanie o statusie `printing`. W przypadku wykrycia aktywnego procesu drukowania, bieżąca iteracja harmonogramu zostaje natychmiast przerwana.
2. **Weryfikacja stanu oczekiwania:** System kontroluje, czy proces nie został wstrzymany przez zadanie oczekujące na potwierdzenie fizycznej gotowości drukarki. Jeżeli takie zadanie istnieje, wybór kolejnej pozycji z kolejki zostaje pominięty.
3. **Wybór i alokacja zasobu:** W przypadku braku blokad, system wyszukuje zadanie o statusie `pending` lub `scheduled` o najwcześniejszej sygnaturze czasowej (`scheduledAt`), a następnie dokonuje atomowej zmiany jego statusu na `waiting_for_printer_ready`.

Na listingu 9 zaprezentowano implementację logiki wyboru oraz rezerwacji zadania, z uwzględnieniem dialektu zapyta SQL specyficznego dla bazy danych *Azure Cosmos DB*.

```
1  const query = '  
2    SELECT TOP 1 * FROM c  
3    WHERE (c.status = 'scheduled' OR c.status = 'pending')  
4    AND (c.scheduledAt <= "${now}"  
5    OR IS_NULL(c.scheduledAt)  
6    OR NOT IS_DEFINED(c.scheduledAt))  
7    ORDER BY c.scheduledAt ASC  
8  ';  
9  const { resources: jobs } = await container.items.query(query).  
    fetchAll();  
10  
11  for (const job of jobs) {  
12    if(!job.id) continue;  
13  
14    job.status = 'waiting_for_printer_ready';  
15    job.waitingStartedAt = nowDate;  
16  
17    await container.item(job.id, job.id).replace(job);  
18  }
```

Listing 9: Implementacja mechanizmu wyboru i rezerwacji zada z kolejki (`startScheduledJobs.ts`)

5.5.3 Monitor gotowoci urzdzienia

Monitor gotowoci odpowiada za stałe nadzorowanie zadań znajdujących się w stanach `waiting_for_printer_ready` oraz `delayed`. Komponent ten realizuje wielopoziomowy, deterministyczny proces przypomnień i eskalacji stanów:

- **Po 60 minutach** — następuje wyzwolenie i wysłanie pierwszego powiadomienia przypominającego.
- **Po 3 godzinach** — następuje wysłanie drugiego powiadomienia przypominającego.
- **Po 12 godzinach** — system dokonuje automatycznej zmiany statusu zadania na `delayed`.
- **Po 48 godzinach** — proces oczekiwania zostaje przerwany, a zadanie otrzymuje status terminalny `cancelled`.

Dystrybucja komunikatów alarmowych i powiadomień realizowana jest za pośrednictwem uniwersalnego mechanizmu sieciowego typu `Webhook`, którego adres docelowy jest definiowany w konfiguracji systemu poprzez zmienną środowiskową `ALERT_WEBHOOK_URL`.

5.5.4 Odbiornik telemetrii

Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, który jest natywnie udostępniany przez usługę *Azure IoT Hub*. W tabeli 6 zestawiono typy zdarzeń telemetrycznych generowanych przez agenta sprzętowego oraz powiązane z nimi akcje modyfikacji dokumentów w bazie danych *Azure Cosmos DB*.

Tabela 6: Zdarzenia telemetryczne przetwarzane przez usług Printer Service.

Zdarzenie	Akcja w bazie danych Azure Cosmos DB
print_started	Zmiana statusu zadania na printing oraz rejestracja sygnatury czasowej w polu startedAt.
print_progress	Sukcesywna aktualizacja metryk postępu w polach progress, printTime oraz printTimeLeft.
print_completed	Zmiana statusu na completed wraz z zapisem czasu zakończenia (completedAt) i całkowitego czasu trwania procesu (printDuration).
print_failed	Zmiana statusu na failed oraz utrwalenie kodu błędu lub przyczyny awarii w polu failureReason.
print_cancelled	Zmiana statusu na cancelled oraz rejestracja momentu przerwania pracy w polu cancelledAt.

5.5.5 Punkty końcowe HTTP API

W tabeli 7 zestawiono i opisano punkty końcowe (ang. *endpoints*) wystawione przez interfejs sieciowy usługi *Printer Service*.

Tabela 7: Punkty końcowe usługi Printer Service.

Metoda	ścieżka	Opis
GET	/printer/jobs	Pobranie spaginowanej listy zadań z obsługą dynamicznego filtrowania wyników.
GET	/printer/jobs/:id	Pobranie szczegółowych parametrów i metryk wybranego zadania.
GET	/printer/jobs/:id/progress	Pobranie bieżących informacji o postępie i szacowanym czasie zakończenia aktywnego procesu druku.

Tabela 7 – ciąg dalszy z poprzedniej strony

Metoda	cieka	Opis
POST	/printer/jobs/:id/cancel	Wymuszenie przerwania i anulowania zadania drukowania (dostęp zastrzeżony dla administratora).
POST	/printer/jobs/:id/retry	Ponowne dodanie wybranego (np. przerwane) zadania do kolejki produkcyjnej.
POST	/printer/jobs/:id/confirm-ready	Zdalne potwierdzenie fizycznej gotowości urządzenia i stanu roboczego przez użytkownika.
DELETE	/printer/jobs/:id	Trwałe usunięcie rekordu zadania z bazy danych (dostęp zastrzeżony dla administratora).
GET	/printer/status	Pobranie aktualnego stanu operacyjnego fizycznej drukarki 3D.
GET	/printer/metrics	Pobranie zagregowanych metryk statystycznych systemowych z podziałem na statusy.
GET	/health	Punkt kontrolny diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

5.6 Video Service

5.6.1 Opis ogólny

Usługa *Video Service* to mikroserwis zaimplementowany w środowisku uruchomieniowym Node.js [8] z wykorzystaniem frameworka Express.js [10] oraz języka JavaScript. Komponent ten odpowiada za tymczasowe przechowywanie aktualnego adresu URL strumienia

wideo z kamery systemu *OctoPrint* w pamięci operacyjnej serwera (ang. *in-memory storage*), co eliminuje konieczność ciągłego odpytywania bazy danych o wolnozmiennie parametry sieciowe.

Agent uruchamiany na mikrokomputerze Raspberry Pi rejestruje bieżący adres kamery (lokalny adres IP lub adres publiczny udostępniany za pośrednictwem tunelu sieciowego *Cloudflare*) po każdym zainicjowaniu systemu operacyjnego. Aplikacja kliencka (frontendowa) cyklicznie pobiera ten adres i renderuje dynamiczny strumień wideo bezpośrednio w interfejsie użytkownika w elemencie `<img>` z wykorzystaniem formatu MJPEG (*Motion JPEG*).

5.6.2 Punkty końcowe API

W tabeli 8 zestawiono i opisano punkty końcowe (ang. *endpoints*) udostępniane przez interfejs sieciowy usługi *Video Service*.

Tabela 8: Punkty końcowe usługi Video Service.

Metoda	ścieżka	Opis
POST	/camera-url	Rejestracja lub aktualizacja nowego adresu URL strumienia wideo z kamery.
GET	/camera-url	Pobranie aktualnego adresu URL kamery w celu poprawnego wyrenderowania podglądu.
GET	/health	Punkt końcowy diagnostyki stanu działania mikroserwisu (ang. <i>health check</i>).

6 Implementacja warstwy prezentacji

6.1 Struktura architektoniczna aplikacji klienckiej

Aplikacja kliencka została zaimplementowana z wykorzystaniem biblioteki *React* [16] w wersji 18 oraz języka TypeScript [6]. Jako środowisko deweloperskie i narzędzie budujące (ang. *bundler*) wykorzystano *Vite* [17], natomiast zarządzanie mapowaniem adresów URL i nawigację realizowane jest przy użyciu biblioteki *React Router* [32] w wersji 6.

Zarządzanie globalnym stanem aplikacji klienckiej (obejmującym dane profilowe zalogowanego użytkownika, bieżący status operacyjny drukarki oraz metryki systemowe) powierzono bibliotece *Zustand* [18]. W celu zachowania ciągłości sesji i preferencji interfejsu (takich jak motyw graficzny czy wybrany język), wykorzystano oprogramowanie pośredniczące *persist*, które automatyzuje proces synchronizacji i trwałego przechowywania danych w pamięci podręcznej *localStorage* przeglądarki internetowej.

Struktura drzewa katalogów aplikacji została zorganizowana w sposób modułowy [7] i przedstawia się następująco:

- `src/components/` — komponenty interfejsu użytkownika wielokrotnego użytku, odpowiadające za renderowanie pojedynczych elementów (m.in. `Card`, `StatusBadge`, `ProgressBar`, `ConfirmDialog`, `Layout`, `LoadingSpinner`),
- `src/pages/` — komponenty wysokiego poziomu reprezentujące autonomiczne widoki aplikacji (m.in. `HomePage`, `LoginPage`, `RegisterPage`, `DashboardPage`, `UploadPage`, `ProfilePage`, `PrintControlPage`, `AdminDashboard`),
- `src/services/api.ts` — implementacja scentralizowanego klienta API odpowiedzialnego za komunikację sieciową z serwerami warstw mikroserwisów,
- `src/store/useStore.ts` — implementacja globalnego magazynu stanu (ang. *store*) biblioteki *Zustand* [18],
- `src/i18n/` — konfiguracja internacjonalizacji aplikacji oparta na bibliotece *i18next* wraz z plikami translacji interfejsu,
- `src/types/` — definicje interfejsów i typów języka TypeScript, w tym struktur `User`, `Job` oraz `PrinterStatus`,

- `src/utils/formatters.ts` — funkcje pomocnicze realizujące operacje bezstanowe, takie jak formatowanie dat, rozmiarów plików czy czasu.

6.2 Scentralizowany klient API

Komunikacja z rozproszoną warstwą backendów realizowana jest przez klasę `ApiClient` zdefiniowaną w module `api.ts`. Klasa ta enkapsuluje i konfiguruje niezależne instancje biblioteki *Axios* [33] dedykowane dla poszczególnych mikroserwisów systemu (*AuthService*, *User Service*, *Printer Service* oraz *Files Service*).

W celu automatyzacji obsługi protokołu HTTP, w konfiguracji instancji zaimplementowano mechanizmy przechwytyjące (ang. *interceptors*). Odpowiadają one za automatyczne wstrzykiwanie tokenu dostępu do nagłówka *Authorization* (w standardzie *Bearer*) dla każdego dania wychodzącego, a także realizują asynchroniczną procedurę odnowienia sesji w przypadku przechwycenia odpowiedzi z kodem błędów HTTP 401 (*Unauthorized*).

Na listingu 10 przedstawiono implementację interceptora odpowiedzi, który w sposób reaktywny zarządza odnowianiem tokenów i ponawianiem pierwotnych danych sieciowych.

```

1 client.interceptors.response.use(
2   response => response,
3   async (error: AxiosError) => {
4     if (error.response?.status === 401) {
5       try {
6         const refreshToken = localStorage.getItem('refreshToken');
7         if (refreshToken) {
8           const { data } =
9             await this.authClient.patch<
10               Omit<AuthResponse, 'user'>
11             >(
12               '/auth/refresh',
13               { refreshToken }
14             );
15             localStorage.setItem('accessToken', data.accessToken);
16             localStorage.setItem('refreshToken', data.refreshToken);
17             if (error.config) {
18               error.config.headers.Authorization = `Bearer ${data.
19               accessToken}`;
20             return axios.request(error.config);

```

```

20     }
21   }
22   } catch {
23     this.logout();
24   }
25 }
26 return Promise.reject(error);
27 }
28 );

```

Listing 10: Implementacja interceptora automatycznego odwiezania tokenów JWT (`api.ts`)

6.3 Mechanizm routingu i ochrona tras

W celu zapewnienia selektywnej kontroli dostępu do zasobów warstwy prezentacji, w aplikacji zaimplementowano mechanizm tras chronionych (ang. *protected routes*) [32]. Wykorzystuje on autorski komponent opakowujący, który dynamicznie weryfikuje stan sesji użytkownika. Dostęp do widoków domenowych wymaga aktywnego uwierzytelnienia, podczas gdy trasy administracyjne realizują dodatkową autoryzację opartą na rolach (RBAC), sprawdzając uprawnienia użytkownika przed wyrenderowaniem komponentów docelowych.

Na listingu 11 zaprezentowano deklaracyjną implementację komponentu `ProtectRoute`, zintegrowanego z globalnym magazynem stanu systemu.

```

1 function ProtectRoute({ children, requireAdmin = false }: {
2   children: React.ReactNode; requireAdmin?: boolean }) {
3
4   const { isAuthenticated, user } = useStore();
5
6   if (!isAuthenticated) {
7     return <Navigate to="/login" replace />;
8   }
9
10  if (requireAdmin && user?.role !== 'admin') {
11    return <Navigate to="/" replace />;
12  }
13  return <>{children}</>;

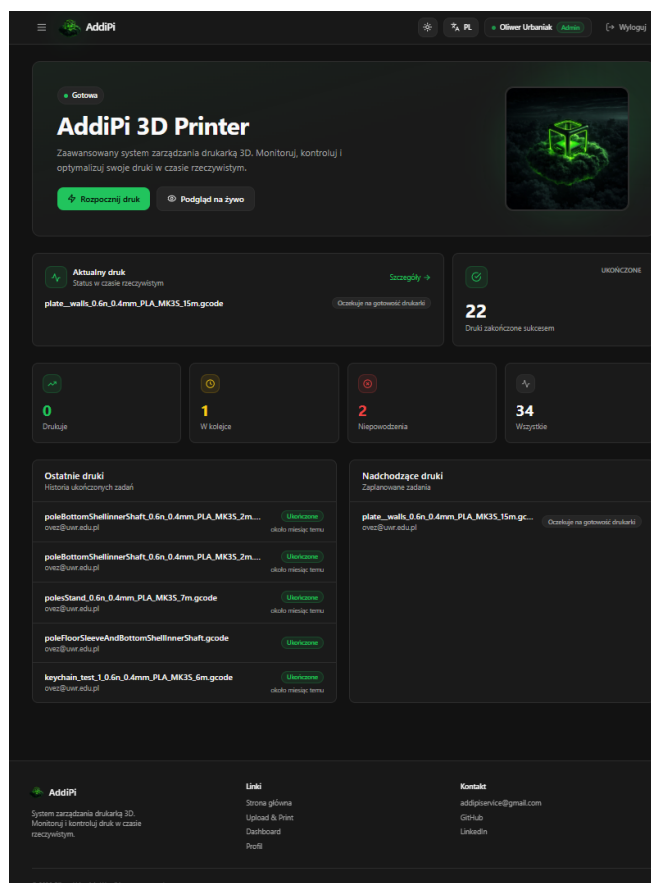
```


Listing 11: Implementacja komponentu tras chronionych (App.tsx)

6.4 Ekran aplikacji klienckiej

6.4.1 Strona główna systemu

Strona główna (ang. *landing page*) agreguje i prezentuje kluczowe parametry operacyjne całego ekosystemu. Na widoku tym wyświetlany jest aktualny status fizycznej drukarki 3D, postęp procentowy i czasowy realizacji bieżącego procesu produkcyjnego, rejestr ostatnio ukończonych wydruków oraz lista nadchodzących zada zaplanowanych w kolejce. W celu zapewnienia wysokiej reaktywności, dane prezentowane w interfejsie są cyklicznie odświeżane z interwałem 5 sekund. Graficzną reprezentację tego ekranu przedstawiono na rysunku 6.

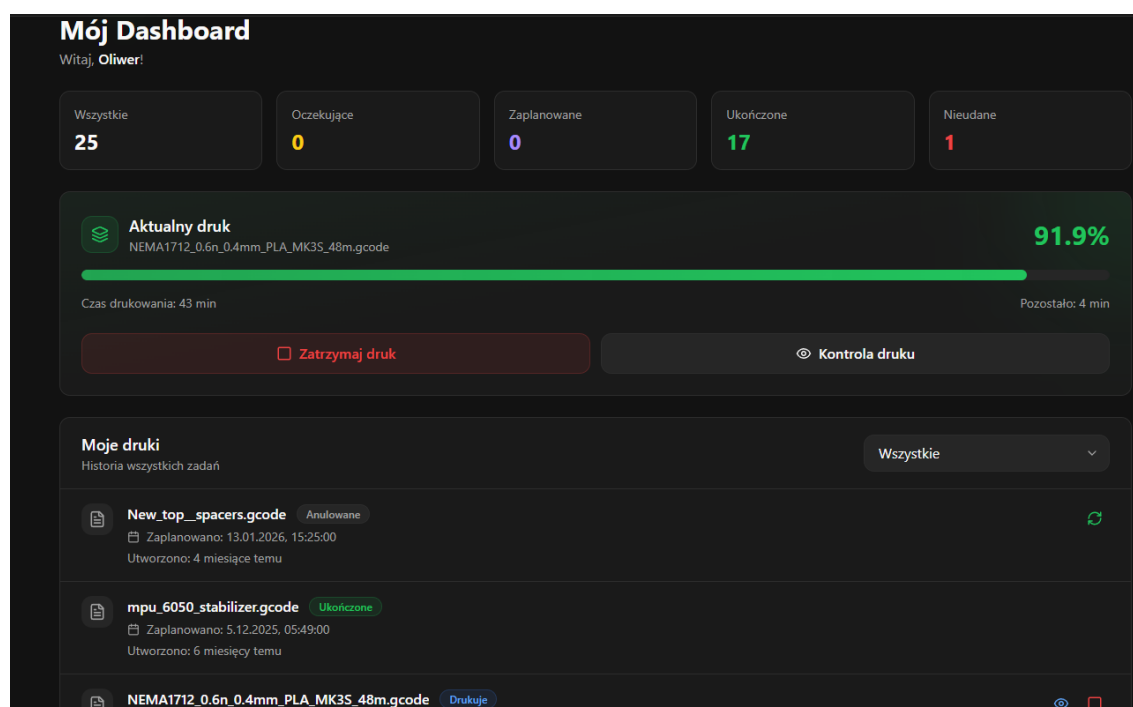
Rysunek 6: Widok strony głównej portalu *AddiPi*.

6.4.2 Panel użytkownika

Panel użytkownika (ang. *User Dashboard*) stanowi spersonalizowane centrum dowodzenia procesami wytwórczymi przypisanymi do danego konta. Widok ten zaprezentowano na rysunku 7. Ekran ten udostpnia:

- zagregowane statystyki zada (liczb przesanych zlece z podziaem na statusy),
- parametry aktywnego procesu drukowania, obejmujce dynamiczny pasek postpu oraz estymowany czas pozostay do zakoczenia operacji,
- kompletn histori druków z moduem wyszukiwania i filtrowania danych.

Z poziomu tego panelu zalogowany operator posiada uprawnienia do manualnego anulowania aktywnego zadania, ponownego zakolejkowania procesu zakoczenia niepowodzeniem (ang. *retry flow*) oraz bezporedniego przejcia do dedykowanego ekranu szczegóowej kontroli procesu drukowania.



Rysunek 7: Widok panelu użytkownika portalu *AddiPi*.

6.4.3 Przesyanie i planowanie zada

Ekran przesysania plików (*UploadPage*), zaprezentowany na rysunku 8, udostpnia użytkownikom intuicyjny interfejs typu *drag-and-drop* (przecignij i upu). Komponent ten

realizuje wstpn walidacj po stronie klienta, weryfikujc poprawne rozszerzenie (.gcode) oraz dopuszczalny rozmiar przesyanego wolumenu.

Widok zosta wyposaony w opcjonalny modu harmonogramowania terminu realizacji zadania, bazujcy na selektorze daty i godziny (ang. *date-time picker*). Interfejs w sposób reaktywny informuje operatora o statusie operacji sieciowej, wywietlajc dedykowane komunikaty o poprawnym przetworzeniu lub bdzie zapytania HTTP.

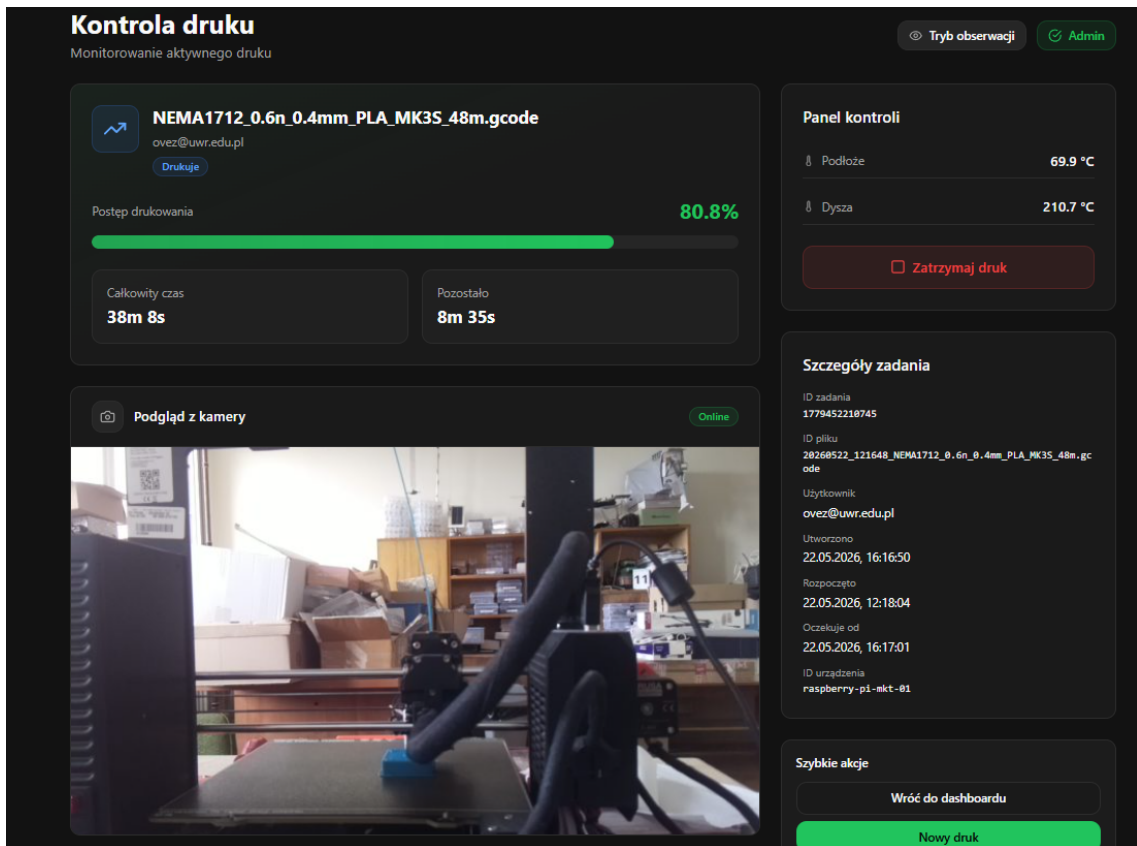
Rysunek 8: Widok panelu przesyania i planowania zada w portalu *AddiPi*.

6.4.4 Kontrola procesu drukowania

Dedykowany ekran kontroli procesu drukowania (*PrintControlPage*), zaprezentowany na rysunku 9, agreguje szczegóowe parametry techniczne aktywnego zlecenia. Widok ten udostpnia dynamiczny pasek postpu, biece odczyty temperatury dyszy ekstrudera oraz stou roboczego, a take strumie wideo z kamery systemu *OctoPrint*. Strumie ten, przesany asynchronicznie w formacie MJPEG, jest renderowany bezporednio w strukturze dokumentu w elemencie `<img>`.

Interfejs zosta wyposaony w panel akcji operacyjnych, który umoliwia uprawnionym uytownikom wstrzymanie lub wznowienie procesu wytwórczego oraz zdalne potwierdze-

nie fizycznej gotowoci urzdzenia do pracy. Poniej sekcji wizualnej prezentowane s metryki i parametry konfiguracyjne powizane z realizowanym zadaniem.



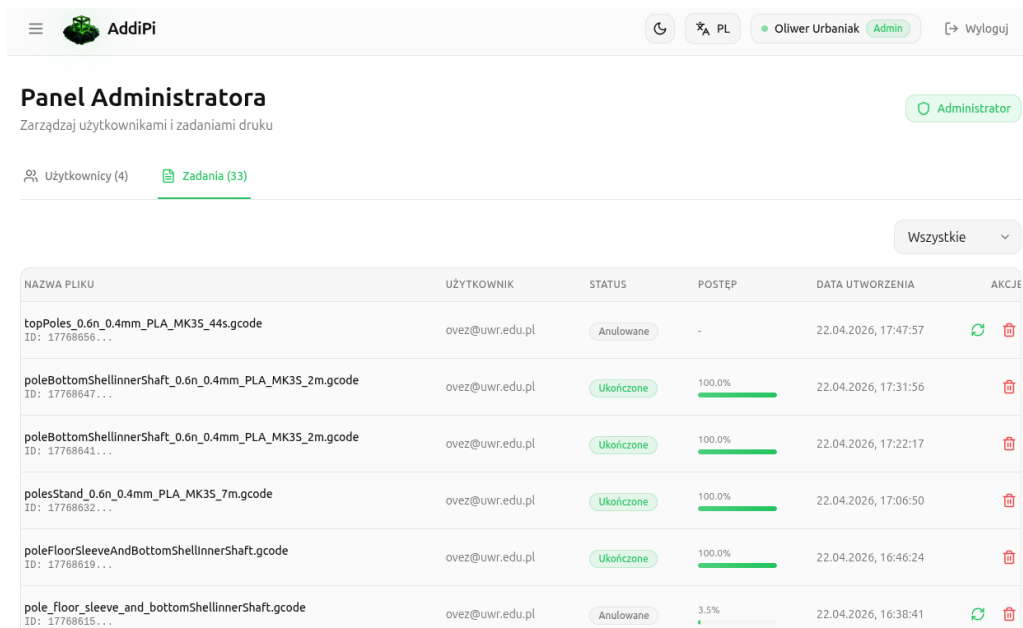
Rysunek 9: Widok panelu kontroli procesu drukowania w portalu *AddiPi*.

6.4.5 Panel administracyjny

Panel administracyjny (ang. *Admin Dashboard*) udostpnia personelowi zarzdzajcemu globalny wgląd w rejestry wszystkich kont uytkowników oraz zada produkcyjnych w systemie. Dane te s prezentowane w postaci ustrukturyzowanych tabel zaimplementowanych z moduami zaawansowanego sortowania, wyszukiwania i filtrowania, co zobrazowano na rysunku 10.

Z poziomu tego widoku administrator posiada uprawnienia do:

- modyfikacji uprawnie i ról uytkowników (nadawanie statusu admin lub user),
- trwalego usuwania profili uytkowników z bazy danych,
- kompleksowego zarzdzania cyklem ycia wszystkich zada w kolejce, wczajc w to ich wymuszone anulowanie, ponowne zakolejkowanie lub usunicie z rejestru.



The screenshot shows the 'Panel Administratora' (Administrator Panel) of the AddiPi system. The header includes the AddiPi logo, a user profile for 'Oliver Urbaniak' with an 'Admin' role, and a language selector set to 'PL'. Below the header, there are tabs for 'Użytkownicy (4)' and 'Zadania (33)', with 'Zadania' being the active tab. A dropdown menu shows 'Wszystkie' (All). The main content is a table with columns: 'NAZWA PLIKU' (File Name), 'UŻYTKOWNIK' (User), 'STATUS' (Status), 'POSTĘP' (Progress), 'DATA UTWORZENIA' (Creation Date), and 'AKCJE' (Actions). The table lists six tasks, each with a file name, user 'ovez@uwr.edu.pl', a status (either 'Anulowane' or 'Ukończone'), a progress bar, a creation date, and action icons (refresh and delete).

NAZWA PLIKU	UŻYTKOWNIK	STATUS	POSTĘP	DATA UTWORZENIA	AKCJE
topPoles_0.6n_0.4mm_PLA_MK35_44s.gcode ID: 17768656...	ovez@uwr.edu.pl	Anulowane	-	22.04.2026, 17:47:57	
poleBottomShellInnerShaft_0.6n_0.4mm_PLA_MK35_2m.gcode ID: 17768647...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:31:56	
poleBottomShellInnerShaft_0.6n_0.4mm_PLA_MK35_2m.gcode ID: 17768641...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:22:17	
polesStand_0.6n_0.4mm_PLA_MK35_7m.gcode ID: 17768632...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 17:06:50	
poleFloorSleeveAndBottomShellInnerShaft.gcode ID: 17768619...	ovez@uwr.edu.pl	Ukończone	100.0%	22.04.2026, 16:46:24	
pole_floor_sleeve_and_bottomShellInnerShaft.gcode ID: 17768615...	ovez@uwr.edu.pl	Anulowane	3.5%	22.04.2026, 16:38:41	

Rysunek 10: Widok panelu administracyjnego portalu *AddiPi*.

6.5 Internacjonalizacja aplikacji

W celu zapewnienia uniwersalności oraz dostępności systemu, interfejs użytkownika został przygotowany w dwóch wersjach językowych: polskiej (ustawionej jako domyślna) oraz angielskiej. Warstwa translacji opiera się na bibliotece *i18next* [34], a same tłumaczenia poszczególnych fraz i etykiet są przechowywane w ustrukturyzowanych plikach formatu JSON (`pl.json` oraz `en.json`), zlokalizowanych w dedykowanym katalogu `src/i18n/locales/`.

Przełączanie wersji językowej realizowane jest w sposób dynamiczny i asynchroniczny, eliminując konieczność ponownego przebudowania struktury dokumentu DOM przez przeglądanie internetu. Jako dowód poprawnego działania mechanizmu umiędzynarodowienia (ang. *internationalization*), na opisanym wcześniej rysunku 8 zaprezentowano widok panelu przesyłania zadań z aktywnym językiem angielskim.

6.6 Zarządzanie motywami graficznymi interfejsu

Aplikacja kliencka w pełni obsługuje dwa warianty wizualne interfejsu użytkownika: motyw jasny (ang. *light mode*) oraz ciemny (ang. *dark mode*). Architektura tego mechanizmu bazuje na natywnych zmiennych CSS (ang. *CSS custom properties*), które zdefiniowano w selektorach `:root` oraz `.dark`. Są one dynamicznie interpretowane i wstrzykiwane przez

silnik frameworka *Tailwind CSS* [19].

Bieca preferencja operatora dotyczyca wybranego schematu kolorów jest utrwalana w pamici podrcznej `localStorage` z wykorzystaniem mechanizmu automatycznej persystencji stanu globalnego biblioteki *Zustand* [18]. Pozwala to na zachowanie wybranego wariantu przy kolejnych sesjach uytkownika. Dla porównania walorów wizualnych, na przytoczonym wczesniej rysunku 10 przedstawiono interfejs panelu administracyjnego z aktywnym motywem jasnym, podczas gdy pozostae ekrany zobrazowano w domylnym wariacie ciemnym.

7 Agent sprztowy — AddiPi Agent

7.1 Opis ogólny

Komponent *AddiPi Agent* stanowi lokaln warstw oprogramowania uruchamian bezpo-
rednio na mikrokomputerze Raspberry Pi. Peń on rol dwukierunkowego pośrednika (ang.
proxy) pomiędzy rozproszon architektur chmur (usugami *Azure IoT Hub* oraz *Azure
Blob Storage*) a niskopoziomym, lokalnym interfejsem sterujcym drukarki 3D, którym
zarzda serwer *OctoPrint* [5].

Agent zosta zaimplementowany w jzyku Python [9]. Architektura operacyjna tego
komponentu zosta zaprojektowana w taki sposób, aby funkcjonowa on jako proces dziaa-
jcy w ptli cigej. W zalenoci od konfiguracji docelowej, jego cyklem ycia zarzda demon
systemowy oprogramowania ukadowego *systemd* lub odizolowany kontener rodowiska
Docker [20].

7.2 Struktura obiektowa i architektura klas

W celu zapewnienia wysokiej modularnoci oraz atwoci rozbudowy kodu ródowego, archi-
tektur agenta podzielono na trzy gówne klasy o cile zdefiniowanych odpowiedzialnociach
(zgodnie z zasad pojedynczej odpowiedzialnoci) [7]:

- **PrinterAgent** — implementuje nadrzdñ logik biznesow aplikacji brzegowej. Od-
powiada za utrzymywanie staego poczenia z usug *Azure IoT Hub*, asynchroniczny
pobór plików sterujcych z repozytorium chmurowego, transmisj strumienia danych
telemetrycznych oraz nadrzdne monitorowanie stanów procesu produkcyjnego.
- **OctoPrintClient** — stanowi warstw abstrakcji nad interfejsem sieciowym REST
API systemu *OctoPrint*. Klasa ta enkapsuluje operacje wejcia-wyjcia odpowiedzialne
za odczyt biecych temperatur i stanów mechanicznych drukarki, buforowanie plików
w pamici lokalnej urzdzienia, a take bezpoednie sterowanie procesem wytwórczym
(rozpoczynanie, wstrzymywanie i anulowanie druku).
- **ConfigLoader** — komponent pomocniczy odpowiedzialny za bezpieczne wczytywa-
nie parametrów startowych, sekretów chmurowych oraz zmiennych rodowiskowych
utrwalonych w pliku konfiguracyjnym `.env`.

7.3 Poczzenie z IoT Hub

Agent sprztowy inicjuje i utrzymuje stałe połączenie z usługą *Azure IoT Hub* [15] za pośrednictwem protokołu MQTT, wykorzystując do tego celu unikalny ciąg uwierzytelniający urządzenia (ang. *device connection string*). Na listingu 12 przedstawiono fragment kodu odpowiedzialny za instancjonowanie asynchronicznego klienta oraz rejestrację funkcji zwrotnej (ang. *callback function*) dedykowanej do obsługi przychodzących danych RPC.

```
1     self.iot_client = IoTHubDeviceClient.  
2         create_from_connection_string(  
3             device_connection_string  
4         )  
5  
6     self.iot_client.connect()  
7  
8     self.iot_client.on_method_request_received = (  
9         self.handle_method_request  
10    )
```

Listing 12: Inicjalizacja klienta usługi Azure IoT Hub (*printer_agent.py*)

7.4 Obsługiwane metody bezpośrednie

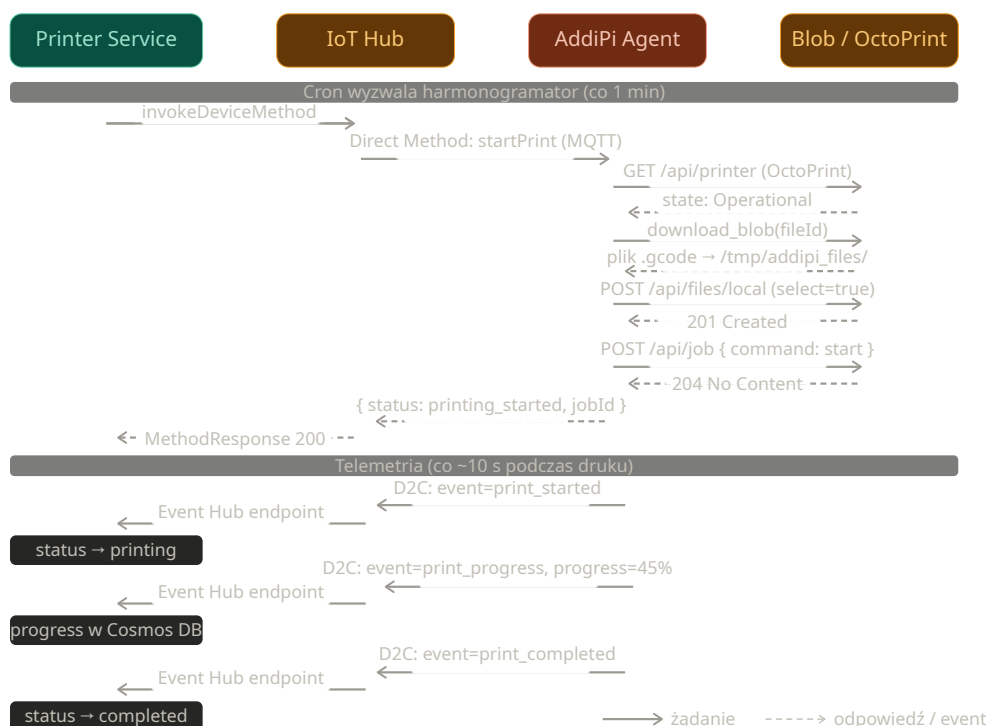
Komponent realizuje zestaw metod bezpośrednich (ang. *Direct Methods*) udostępnianych przez chmur obliczeniową. W tabeli 9 wyszczególniono metody zaimplementowane w strukturze agenta wraz z ich parametrami wejściowymi oraz opisem wywoływanych akcji.

Tabela 9: Metody bezpośrednie obsługiwane przez agenta sprztowego.

Metoda	Parametry wejściowe	Opis operacji systemowej
startPrint	fileId, jobId	Pobranie pliku z Azure Blob Storage, przesłanie go do <i>OctoPrint</i> oraz uruchomienie procesu drukowania
cancelPrint	—	Anulowanie aktywnego procesu wytwórczego za pośrednictwem interfejsu API systemu <i>OctoPrint</i>
getStatus	—	Pobranie aktualnego stanu operacyjnego drukarki, postępu procentowego zadania oraz odczytów termicznych urządzenia

7.5 Przebieg obsługi metody bezporedniej startPrint

Inicjalizacja procesu wytwórczego przez system chmurowy opiera si na synchronicznym wywoaniu metody bezporedniej startPrint. Na rysunku 11 zaprezentowano diagram sekwencji obrazujcy przepływ sterowania oraz interakcje pomidzy agentem sprztowym, uslugami chmurowymi a serwerem *OctoPrint*.



Rysunek 11: Diagram sekwencji obsługi metody bezporedniej startPrint przez *Azure IoT Hub*.

Szczegółowy proces obsługi omawianego dania realizowany jest sekwencyjnie według następującego scenariusza:

1. Agent odbiera dane *Direct Method* zawierające parametry `fileId` oraz `jobId`.
2. Następuje sprawdzenie stanu operacyjnego oraz gotowości mechanicznej drukarki 3D przy użyciu metody `OctoPrintClient.is_printer_ready()`.
3. Plik G-code o wskazanym identyfikatorze jest pobierany z *Azure Blob Storage* i buforowany w lokalnym katalogu tymczasowym `/tmp/addipi_files/`.
4. Agent przesyła pobrany plik do systemu *OctoPrint* poprzez wywołanie punktu koco-

wego POST /api/files/local z parametrem select=true. (co wymusza automatyczny wybór pliku do druku).)

5. Wyzwolenie fizycznego procesu drukowania realizowane jest poprzez zapytanie do punktu kocowego POST /api/job z poleceniem strukturalnym start.
6. Agent wysyła zdarzenie telemetryczne print_started do usługi *Azure IoT Hub*.

7.6 Transmisja danych telemetrycznych

W trakcie aktywnego procesu produkcyjnego, agent sprztowy w sposób ciągły monitoruje stan urządzenia w ramach dedykowanej ptli sprawdzającej. Dane telemetryczne są transmitowane do usługi *Azure IoT Hub* cyklicznie z interwałem 10 sekund. Struktura oraz format przesyłanego komunikatu w formacie JSON zostały szczegółowo zaprezentowane na listingu 13.

```
1  {
2    "event": "print_progress",
3    "timestamp": "2025-12-01T14:30:00.000000",
4    "deviceId": "raspberrypi-mkt-01",
5    "jobId": "1764722864093",
6    "progress": 45.5,
7    "printTime": 1800,
8    "printTimeLeft": 2200,
9    "state": "Printing"
10 }
```

Listing 13: Format struktury wiadomości telemetrycznej transmitowanej do chmury.

7.7 Niskopoziomowa komunikacja z serwerem OctoPrint

Klasa OctoPrintClient hermetyzuje i realizuje komunikację sieciową z systemem *OctoPrint* [5] w ramach lokalnej sieci komputerowej (z wykorzystaniem lokalnego mapowania adresów, np. <http://prusai3mk3.local>). Autoryzacja i bezpieczeństwo zapytań HTTP są zapewniane poprzez wstrzykiwanie unikalnego klucza dostępowego do nagłówka X-API-Key każdego zapytania.

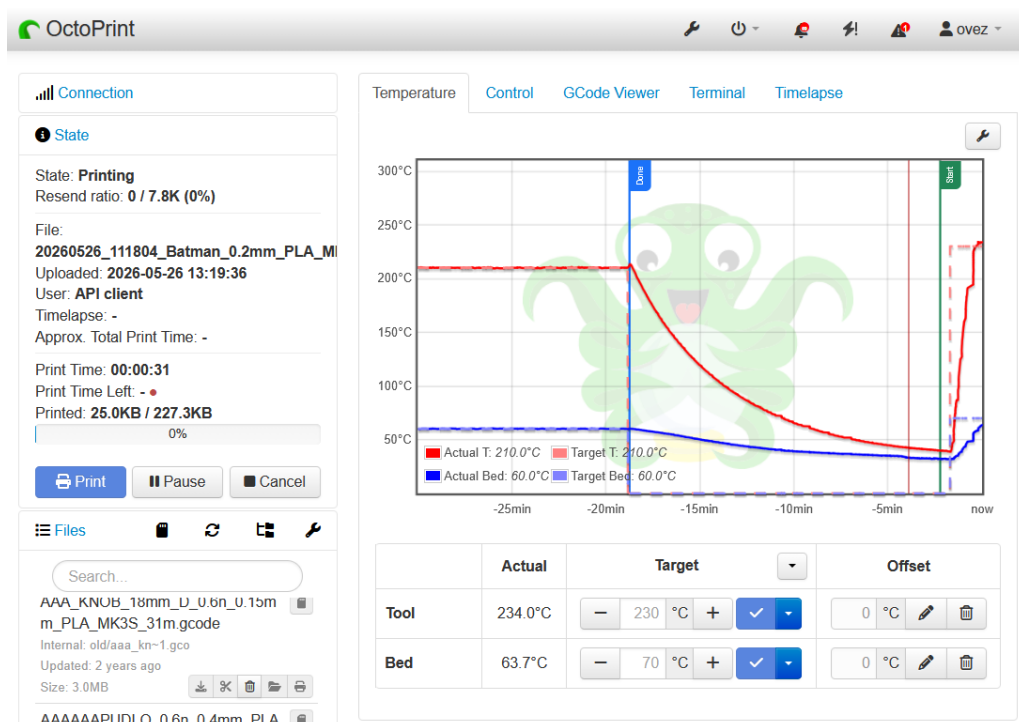
Na listingu 14 zestawiono kluczowe operacje zaimplementowane w strukturze omawianej klasy wraz z przypisanymi do nich punktami kocowymi (ang. *endpoints*) oraz

strukturami polece lokalnego interfejsu REST API.

```
1 // Definicja kluczowych metod wchodzących w skład klasy
   OctoPrintClient:
2
3 get_printer_state()      ---> GET /api/printer
4 upload_and_select_file() ---> POST /api/files/local
5 start_print()           ---> POST /api/job { "command": "
   start" }
6 cancel_print()          ---> POST /api/job { "command": "
   cancel" }
7 get_job_info()          ---> GET /api/job
```

Listing 14: Struktura metod klasy OctoPrintClient i ich mapowanie na punkty kcowe API.

W celu zobrazowania rodowiska niskopoziomowego, z którym bezporędnio integruje si opisany agent sprztowy, na rysunku 12 przedstawiono natywny panel graficzny oprogramowania serwerowego *OctoPrint*. Widok ten odzwierciedla stan aktywnego procesu drukowania, z którego klasa OctoPrintClient asynchronicznie parsuje metryki czasu, postpu oraz biece parametry termiczne dyszy i stou roboczego.



Rysunek 12: Interfejs systemu *OctoPrint* z widokiem aktywnego procesu druku.

8 Projekt mechaniczny obudowy i pozycjonowania zestawu

8.1 Cel i zakres projektu mechanicznego

Kompletna implementacja systemu *AddiPi* — poza warstw stricte programow — wymaga fizycznego osadzenia mikrokomputera Raspberry Pi w bezporednim ssiedztwie urzdzienia wytwórczego wraz z integracj moduu kamery cyfrowej monitorujcej stó roboczy drukarki 3D.

W ramach niniejszej pracy inzynierskiej opracowano, a nastpnie wytworzono technologii przyrostow (ang. *Additive Manufacturing*) [1] dedykowany zestaw komponentów mechanicznych. Zosta on podzielony na dwa główne moduly strukturalne:

- **CASE** — obudowa ochronna mikrokomputera Raspberry Pi zintegrowana z gniazdem montaowym moduu kamery brzegowej,
- **STAND** — regulowane rami pozycjonujce (uchwyt), umoliwiajce swobodn zmian kta widzenia i stabilizacj zestawu wzgldem pola roboczego drukarki 3D.

Trójkwymiarowe modele geometryczne (CAD) opracowano w programie *SolidWorks* [23] (format .SLDPRT dla pojedynczych czci oraz .SLDASM dla zoe).

W celu zapewnienia otwartoci projektu, w repozytorium zasobów systemu — obok natywnych formatów oprogramowania *SolidWorks* — umieszczono modele w neutralnym formacie wymiany danych .STEP, umoliwiajcy dalsz edycj w innych rodowiskach CAD, a take pliki .3MF i .gcode przygotowane do wydruku na drukarce Prusa MK3S wyposaonej w dysz o rednicy 0,6 mm z wykorzystaniem filamentu PLA.

8.2 Modu CASE — obudowa Raspberry Pi z kamer

8.2.1 Podstawa projektowa

Modu CASE nie zosta zaprojektowany od podstaw — jako baz konstrukcyjn wykorzystano otwarty projekt udostpniony w serwisie *GrabCAD* przez *219 Design* [35], opublikowany na licencji **CC BY 4.0**.

Oryginalny model geometryczny dedykowany by dla platformy **Raspberry Pi 4 Model B**. Ze wzgldu na zastosowanie w systemie *AddiPi* urzdzienia **Raspberry Pi 3 Model B**, konieczne byo wprowadzenie szeregu modyfikacji konstrukcyjnych.

8.2.2 Zakres modyfikacji geometrycznych

Głównym zadaniem w ramach adaptacji modelu było przeprojektowanie komponentu BottomShell, stanowiącego korpus obudowy.

Najistotniejsze różnice konstrukcyjne, które determinowały konieczność modyfikacji makra bryowego obudowy, obejmowały następujące aspekty:

- **Interfejs zasilania** — zmian standardu i lokalizacji złącza zasilającego z formatu USB-C (stosowanego w nowszej architekturze generacji czwartej) na gniazdo micro-USB, charakterystyczne dla wersji Raspberry Pi 3,
- **Układ portów USB-A i LAN** — odwrócenie i przesunięcie pozycji bloków złączy USB oraz gniazda ethernetowego RJ-45 względem krawędzi płytki PCB,
- **Złącza peryferyjne i multimedialne** — całkowite zmiany położenia oraz gabarytów pinowego portu HDMI i dedykowanego złącza tamy interfejsu kamery (CSI).

Projekt geometryczny uwzględnia fakt, że obie platformy charakteryzują się identycznymi wymiarami zewnętrznymi laminatu bazowego ($85,0 \times 56,0$ mm) oraz zachowują ten sam standard rozstawu otworów montażowych ($58,0 \times 49,0$ mm, dedykowany dla rur i tulei dystansowych w rozmiarze M2.5). Pozwoliło to na zachowanie oryginalnej struktury geometrycznej górnej pokrywy obudowy oraz elementów mocujących.

Proces modyfikacji modelu BottomShell w środowisku *SolidWorks* polega na usunięciu dotychczasowych operacji wycięcia i zastąpieniu ich nowymi operacjami szkicu, sparametryzowanymi pod kątem rzeczywistego rozmieszczenia złączy Raspberry Pi 3.

8.2.3 Integracja strukturalna i mocowanie modułu kamery brzegowej

Obudowa CASE została wyposażona w zintegrowany uchwyt modułu kamery Raspberry Pi (w wersji v1 lub v2, wyposażonej odpowiednio w sensor *OV5647* lub *IMX219*).

Uchwyt pozycjonuje kamerę pod stałym kątem skierowanym w stronę pola roboczego drukarki 3D. Tę sygnałowa interfejsu CSI prowadzona jest wewnątrz obudowy za pomocą dedykowanego kanału kablowego.

8.2.4 Struktura plików i organizacja zasobów moduu CASE

W celu zapewnienia przejrzystości oraz uatwienia ewentualnej replikacji lub dalszej modyfikacji projektu mechanicznego, zasoby cyfrowe powizane z moduem CASE zostay ustrukturyzowane w dedykowanym repozytorium. W tabeli 10 zestawiono struktur katalogów oraz opisano zawarto poszczególnych zbiorów danych.

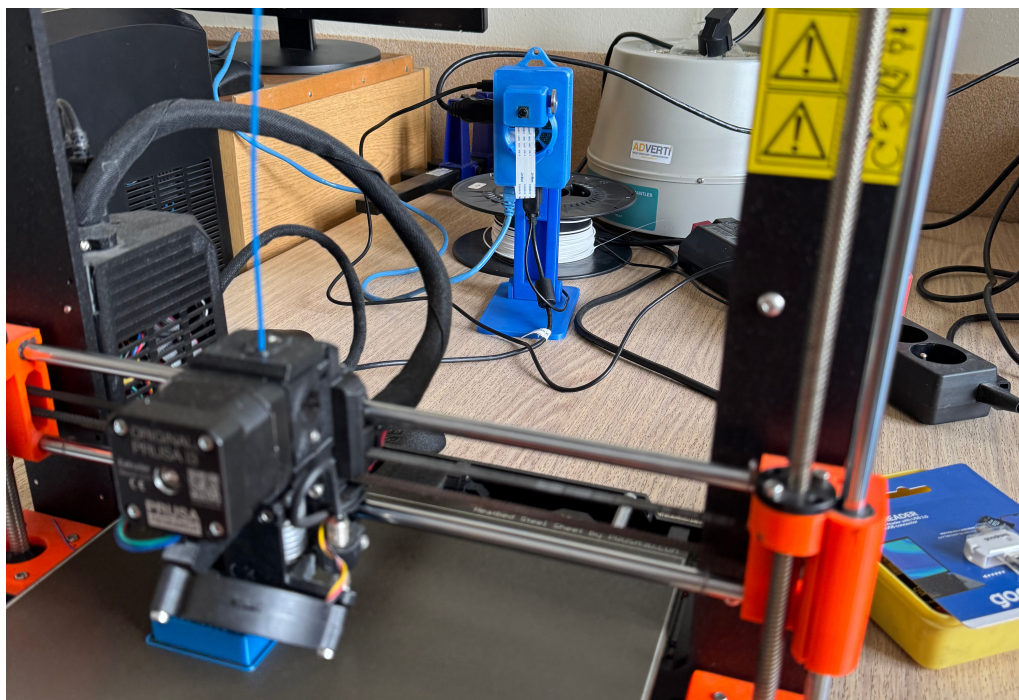
Tabela 10: Struktura katalogów i plików produkcyjnych moduu CASE

Katalog / plik	Zawarto
CASE/files/	Edytowalne modele SolidWorks (.SLDPRT, .SLDASM) oraz modele w formacie wymiany .STEP
CASE/gcode/	Wygenerowane pliki G-code przygotowane dla drukarki Prusa MK3S z dysz 0,6 mm i filamentem PLA
CASE/Ready To Print/	Modele .3MF oraz .STL przygotowane do otwarcia w programie PrusaSlicer
CASE/grabcad/	Oryginalne pliki referencyjne projektu GrabCAD [35] dla Raspberry Pi 4

8.3 Modu STAND — regulowany mechanizm pozycjonujcy

Modu STAND stanowi zaprojektowany od podstaw, autorski mechanizm wsporczy, umoliwiajcy precyzyjne pozycjonowanie obudowy CASE wraz z kamer wzgldem ramy kinematycznej drukarki 3D.

Konstrukcja ramienia opiera si na poczeniach przegubowych i zapewnia dwa stopnie swobody. Ukad ten pozwala na pynn regulacj kta nachylenia osi optycznej obiektywu w paszczynie pionowej oraz dostosowanie wysokoci punktu akwizycji obrazu. Dziki temu moliwe jest optymalne dopasowanie pola widzenia (ang. *Field of View*, FOV) do specyfiki geometrii przestrzeni roboczej danego urzdzienia. Fizyczne wdroenie gotowego zestawu montaowego zamontowanego przy drukarce 3D przedstawiono na rysunku 13.



Rysunek 13: Gotowa konstrukcja mechaniczna zainstalowana przy urządzeniu wytwórczym.

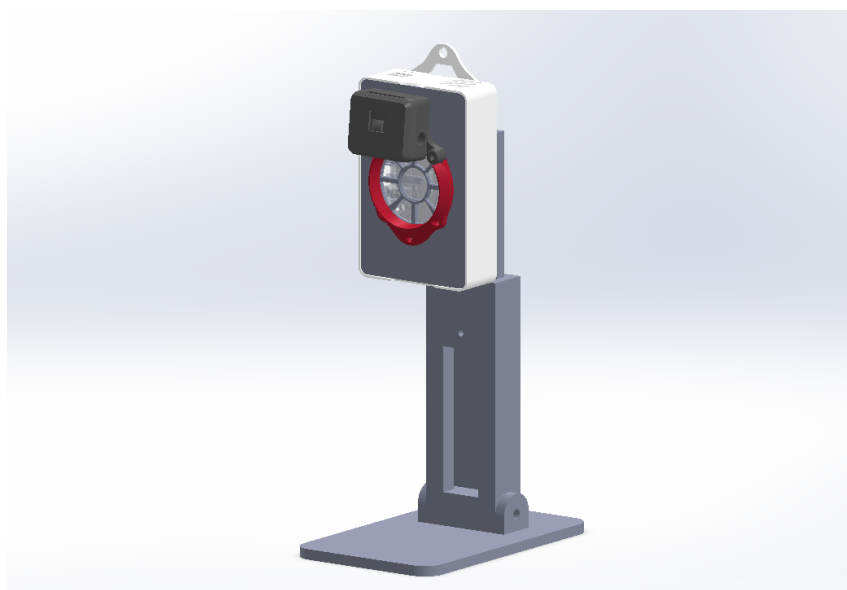
Parametryczne modele ródowe rodowiska *SolidWorks* s przechowywane w dedykowanym katalogu `STAND/own parts/`. Z kolei pliki przygotowane do bezpośredniego procesu przygotowania druku (w formatach `.3MF` oraz `.STL`) zlokalizowano w folderze `STAND/Ready To Print/`, a skompilowane instrukcje sterujące G-code umieszczono w katalogu `STAND/gcode/`.

8.4 Zoenie kompletnego modelu geometrycznego

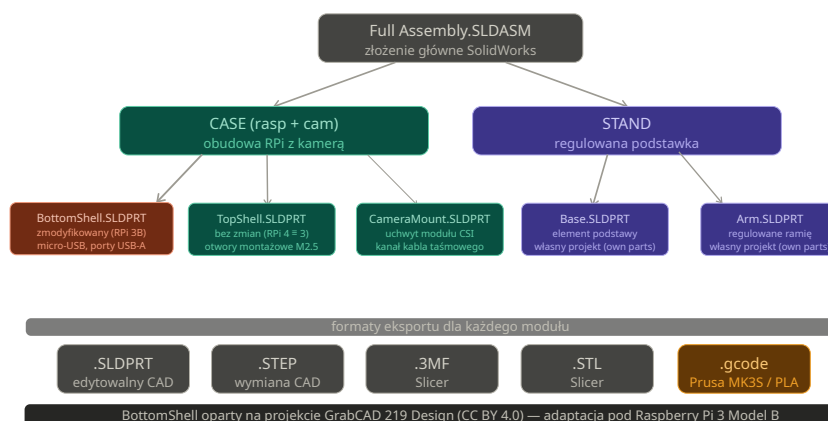
Plik ródowy `CAD/Full Assembly.SLDASM` zawiera kompletne zoenie (ang. *assembly*) obu zaprojektowanych moduów (`CASE` oraz `STAND`) wraz z zaimportowanym modelem mikrokomputera Raspberry Pi, penicym funkcj komponentu referencyjnego. Definicja relacji montaowych (ang. *mates*) wewntz struktury modelu umoliwia weryfikacj poprawności dopasowania tolerancji wymiarowych, przeprowadzenie analizy potencjalnych kolizji geometrycznych oraz kontrol parametrów montaowych przed rozpoczęciem docelowego procesu wytwarzania przyrostowego.

Wizualizacj trójwymiarow tak przygotowanego rodowiska projektowego przedstawiono na rysunku 14. Z kolei na rysunku 15 zaprezentowano strukturalny schemat hierarchii plików i formatów eksportu, który szczegółowo obrazuje architektur całego pakietu mechanicznego — od parametrycznych plików ródowych czci, przez faz przygotowania

wydruku (slicing), a po koczowy kod maszynowy.



Rysunek 14: Trójwymiarowy widok zoenia moduów CASE oraz STAND w rodowisku *SolidWorks* [23].



Rysunek 15: Struktura hierarchiczna zoe, komponentów oraz formatów produkcyjnych moduów *SolidWorks* [23].

8.5 Parametry wydruku

W celu zapewnienia powtarzalności geometrycznej, odpowiedniej wytrzymałości mechanicznej struktury oraz optymalizacji czasu produkcji elementów obudowy, zdefiniowano zestaw rekomendowanych parametrów procesu technologicznego [1]. W tabeli 11 zestawiono szczegółowe nastawy CAM przygotowane dla rodowiska programu typu slicer.

Tabela 11: Zalecane parametry druku 3D elementów obudowy systemu.

Parametr	Warto
Materia	PLA
rednica dyszy	0,6 mm
Wysoko warstwy	0,2 mm (profil <i>Quality</i>)
Wypienienie	20% (wzór Gyroid)
Podpory	Niewymagane dla wikszoci modeli
Orientacja wydruku	Zgodna z konfiguracj w plikach .3MF
Drukarka referencyjna	Prusa MK3S

8.6 Procedura montau zestawu mechanicznego

Proces montau oraz kalibracji przestrzennej zestawu komponentów mechanicznych realizowany jest sekwencyjnie wedug nastpujcego scenariusza:

1. **Wytworzenie komponentów:** Wydrukowanie elementów skadowych obudowy i ramienia na podstawie plików sterujcych .gcode lub projektów technologicznych .3MF oraz .STL zlokalizowanych w katalogach Ready To Print.
2. **Instalacja elektroniki:** Zamontowanie mikrokomputera Raspberry Pi w dolnej czci obudowy (BottomShell) z wykorzystaniem czterech rub montaowych M2.5 oraz dedykowanych nakrtek.
3. **Integracja magistrali CSI:** Podczenie tamy sygnaowej interfejsu CSI do moduu kamery oraz precyzyjne przeprowadzenie jej przez dedykowany przepust osonowy w korpusie obudowy.
4. **Asemlacja korpusu:** Zamkniecie obudowy poprzez naoenie górnej pokrywy (TopShell) oraz mechaniczne zabezpieczenie konstrukcji za pomoc zintegrowanych zatrzasków geometrycznych.
5. **Poczenie moduów:** Osadzenie zmontowanego moduu CASE na regulowanym ramieniu wsporczym STAND wraz z nastawieniem wymaganego kta pochylenia osi optycznej.
6. **Kalibracja pola widzenia:** Ustawienie kompletnego zestawu w ssiedztwie drukarki 3D w sposób zapewniajcy bezkolizyjn prac maszyn oraz pen akwizycj geometrycznego pola roboczego urzdzienia.

9 Konfiguracja Raspberry Pi

9.1 Opis rodowiska

Mikrokomputer *Raspberry Pi 3 Model B* pełni funkcję wza brzegowego systemu *AddiPi*. Odpowiada on za trzy kluczowe zadania:

- uruchamianie serwera *OctoPrint* komunikującego się z drukarką przez interfejs USB,
- wykonywanie kodu agenta *AddiPi Agent* zaimplementowanego w języku Python,
- udostępnianie strumienia wideo z kamery CSI do zewnętrznej usługi *Video Service*.

Bazy systemowej dostarcza *OctoPi* [36] — oficjalny, minimalistyczny obraz *Raspberry Pi OS Lite* z preinstalowanym rodowiskiem *OctoPrint*. Interpreter *Python 3* oraz menedżer *pip* są w tej dystrybucji dostępne domylnie, co wyeliminowało potrzebę konfiguracji rodowiska bazowego.

9.2 Instalacja agenta

Procedura instalacji komponentu *AddiPi Agent* sprowadza się do pobrania zasobów z repozytorium systemowego, wdrożenia pakietów zaleanych oraz konfiguracji zmiennych środowiskowych. Sekwencji wymaganych poleceń powłoki systemu operacyjnego zaprezentowano na listingu 15.

```
1  # Klonowanie repozytorium projektu
2  git clone https://github.com/AddiPi/AddiPi-Agent.git
3  cd AddiPi-Agent
4
5  # Instalacja zależności Python
6  python3 -m pip install -r requirements.txt
7
8  # Konfiguracja zmiennych środowiskowych
9  cp .env.example .env
10 nano .env
11
12 # Uzupelnienie wymaganych parametrów:
13 # DEVICE_CONNECTION_STRING, STORAGE_CONN
```

```
14 # OCTOPRINT_API_KEY, OCTOPRINT_URL
```

Listing 15: Instalacja i inicjalizacja agenta na platformie Raspberry Pi.

9.3 Automatyzacja uruchamiania za pomoc systemd

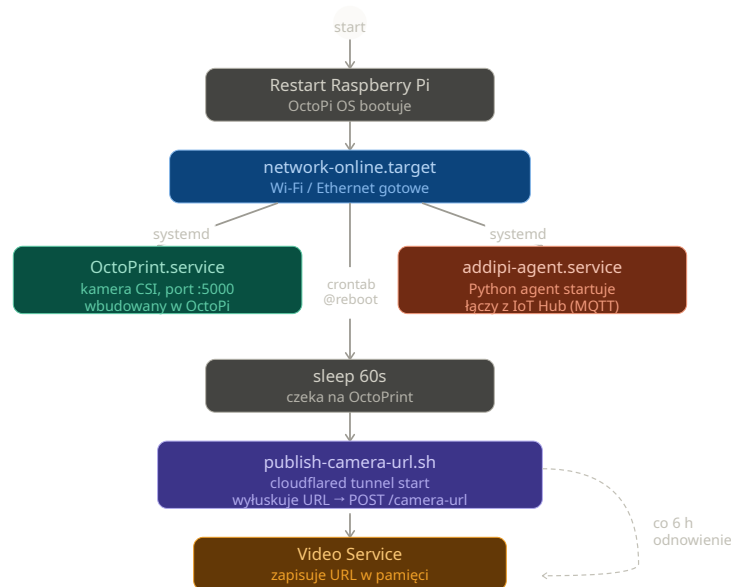
W celu zapewnienia bezobsługowości systemu brzegowego i automatycznego restartu aplikacji po awarii lub restarcie mikrokomputera, skonfigurowano dedykowaną usługę *systemd*. Plik konfiguracyjny jednostki (ang. *unit file*) zlokalizowano w ciele `/etc/systemd/system/addipi-agent.service`, a jego strukturę zaprezentowano na listingu 16.

```
1 [Unit]
2 Description=AddiPi Printer Agent
3 After=network-online.target
4 Wants=network-online.target
5
6 [Service]
7 Type=simple
8 User=pi
9 WorkingDirectory=/home/pi/AddiPi-Agent
10 ExecStart=/usr/bin/python3 src/app.py
11 Restart=always
12 RestartSec=10
13 EnvironmentFile=/home/pi/AddiPi-Agent/.env
14
15 [Install]
16 WantedBy=multi-user.target
```

Listing 16: Plik jednostki konfiguracji `addipi-agent.service` dla menedżera *systemd*.

Dyrektywa `After=network-online.target` gwarantuje uruchomienie skryptu dopiero po poprawnym zainicjowaniu stosu sieciowego. Zapobiega to błądowi gniazda sieciowych (ang. *sockets*) podczas próby nawiązania połączenia przez protokół MQTT z usługą *Azure IoT Hub*. Aktywację i uruchomienie demona realizuje się sekwencyjnie za pomocą poleceń systemowych: `sudo systemctl daemon-reload`, `sudo systemctl enable addipi-agent` oraz `sudo systemctl start addipi-agent`. Bieżący stan procesu weryfikowany jest instrukcją `sudo systemctl status addipi-agent`.

Kompletna sekwencja zdarzeń oraz założone potoki rozruchowych usług systemowych i skryptów harmonogramu po restarcie platformy sprzętowej przedstawiono na rysunku 16.



Rysunek 16: Diagram potoku rozruchowego mikrokomputera po restarcie systemu.

9.4 Udostępnianie strumienia wideo przez Cloudflare Tunnel

Lokalny strumień wideo MJPEG, generowany jest pod adresem:

`http://localhost:8080/?action=stream`

Poczenie to jest domylnie zablokowane poza strukturę sieci lokalnej. W celu umożliwienia zewnętrznego podglądu z kamery w aplikacji klienckiej (hostowanej na platformie *Vercel*), zastosowano technologię *Cloudflare Tunnel* (*cloudflared*) [37].

Narzędzie *cloudflared* zestawia szyfrowane, bezpieczne połączenie wychodzące pomiędzy mikrokomputerem a infrastrukturą *Cloudflare*. Pozwala to na bezpłatne i stabilne zmapowanie lokalnego portu na publiczny adres URL w domenie *trycloudflare.com* (np. `https://<losowy-hash>.trycloudflare.com`) bez konieczności konfiguracji publicznego adresu IP na urządzeniu brzegowym czy modyfikacji zapór sieciowych.

9.4.1 Instalacja narzędzia cloudflared

Wdrożenie oficjalnego klienta *cloudflared* [37] na platformie sprzętowej z procesorem o architekturze ARM wymaga pobrania skompilowanego pliku binarnego, nadania mu

uprawnie wykonywania oraz relokacji do katalogu systemowego. Operacje te realizuje się sekwencyjnie w terminalu systemu operacyjnego za pomocą następujących instrukcji:

```
wget https://github.com
chmod +x cloudflared-linux-arm64
sudo mv cloudflared-linux-arm64 /usr/local/bin/cloudflared
cloudflared -version
```

Ostatnie polecenie wykonuje weryfikację poprawności instalacji poprzez zwrócenie bieżącej wersji oprogramowania w konsoli.

9.4.2 Skrypt automatyzacji publikacji adresu sieciowego

Po uruchomieniu tunelu Cloudflare usługa `cloudflared` przydziela nowy, losowy publiczny adres URL. Aby serwis Video Service posiadał zawsze aktualny adres strumienia kamery, opracowano skrypt `publish-camera-url.sh`, który realizuje następujące operacje:

1. Uruchomienie tunelu `cloudflared tunnel` w tle oraz przekierowanie logów do pliku tymczasowego.
2. Odczekanie kilku sekund na zestawienie połączenia tunelowego.
3. Odczytanie przydzielonego adresu URL z pliku logów na podstawie wzorca domeny `trycloudflare.com`.
4. Przesanie uzyskanego adresu URL do serwisu Video Service za pomocą dania POST `/camera-url`.

Kompletny kod źródłowy tego mechanizmu zaprezentowano na listingu 17.

```
1  #!/bin/bash
2  LOG_FILE="/tmp/cloudflared.log"
3  VIDEO_SERVICE_URL="http://azure.com"
4  CAMERA_LOCAL_URL="http://localhost:8080"
5
6  # Uruchomienie tunelu w tle i przekierowanie strumieni danych
7  cloudflared tunnel --url "$CAMERA_LOCAL_URL" > "$LOG_FILE" 2>&1 &
8
9  # Iteracyjne sondowanie logu w celu ekstrakcji adresu (maks. 30 s
   )
```

```

10 TUNNEL_URL=""
11 for i in $(seq 1 15); do
12     sleep 2
13     TUNNEL_URL=$(grep -E -o 'https://[a-zA-Z0-9-]*\.trycloudflare\.
14         com' "$LOG_FILE" | head -1)
15     if [ -n "$TUNNEL_URL" ]; then
16         break
17     fi
18 done
19
20 if [ -z "$TUNNEL_URL" ]; then
21     exit 1
22 fi
23
24 # Transmisja adresu do centralnej usugi Video Service
25 curl -s -X POST "$VIDEO_SERVICE_URL/camera-url" \
26     -H "Content-Type: application/json" \
27     -d "{\"url\": \"$TUNNEL_URL\"}"

```

Listing 17: Skrypt automatycznej rejestracji dynamicznego adresu URL tunelu (publish-camera-url.sh).

9.4.3 Automatyzacja odnawiania adresu w harmonogramie cron

Z uwagi na fakt, że bezpłatne tunele w domenie trycloudflare.com wykazują tendencję do samoczynnego wygasania po okresie bezczynności lub wskutek fluktuacji sieciowych, wdrożono mechanizm ciągłego odnawiania sesji. Do tego celu wykorzystano systemowy harmonogram zadań *cron*. W tablicy konfiguracji użytkownika (*crontab*) zdefiniowano dyrektywę *@reboot*, która inicjuje skrypt *publish-camera-url.sh* bezpośrednio po starcie systemu operacyjnego, oraz regułę czasową uruchamiając procedurę cyklicznie z interwałem szestiogodzinnych iteracji, co zapewnia ciągłą dystrybucję strumienia wideo.

9.5 Konfiguracja harmonogramu zadań *crontab*

Zarządzanie potokami cyklicznymi oraz automatyzacja skryptów na platformie Raspberry Pi powierzono systemowemu demonowi *cron*. Edycja harmonogramu realizowana jest poleceniem *crontab -e* w kontekście uprawnień użytkownika *pi*. Wprowadzone wpisy

konfiguracyjne, implementujące opóźniony start rozruchowy oraz cykliczne odnawianie tunelu, przedstawiono na listingu 18.

```
1  # Publikacja adresu URL kamery po restarcie systemu (opóźnienie
    60 s)
2  @reboot sleep 60 && /home/pi/scripts/publish-camera-url.sh >> /
    home/pi/logs/camera-url.log 2>&1
3
4  # Odnawianie adresu URL kamery cyklicznie co 6 godzin
5  0 */6 * * * /home/pi/scripts/publish-camera-url.sh >> /home/pi/
    logs/camera-url.log 2>&1
```

Listing 18: Konfiguracja zada harmonogramu cron dla użytkownika pi.

9.5.1 Uzasadnienie zastosowanych parametrów

- `@reboot sleep 60` opóźnienie wynoszące 60 sekund po uruchomieniu systemu jest wymagane, ponieważ środowisko *OctoPi* [36] podczas startu inicjalizuje wiele usług systemowych, m.in. *OctoPrint*, obsług kamery oraz interfejs sieciowy Wi-Fi. Próba uruchomienia tunelu na wcześniejszym etapie może zakończyć się błądem z powodu niedostępności lokalnego portu strumienia wideo.
- `0 */6 * * *` cykliczne uruchamianie skryptu co 6 godzin umożliwia weryfikację dostępności tunelu *Cloudflare* oraz ponowną publikację adresu URL w przypadku zerwania połączenia. Wybrany interwał stanowi kompromis pomiędzy częstotliwością odnawiania a ograniczeniem zbędnego obciążenia sieci i usług zewnętrznych.

9.6 Problem dostępności sieci podczas uruchamiania systemu

W trakcie testów zaobserwowano, że zadania uruchamiane przez mechanizm `@reboot` w *crontab* mogą zostać wykonane przed pełną inicjalizacją interfejsu sieciowego. W rezultacie skrypt publikujący adres URL kamery mógł uruchomić się przed uzyskaniem dostępu do sieci lub przed pełnym uruchomieniem usługi *OctoPrint*.

W przypadku agenta *AddiPi* problem rozwiązano z wykorzystaniem dyrektywy `After=network-online.target` w konfiguracji usługi *systemd*. Dla skryptu uruchamianego przez cron zastosowano dodatkowe opóźnienie (`sleep 60`), stanowiące proste i niezawodne obejście problemu kolejności startu usług.

Alternatywnym rozwiązaniem byłoby zarejestrowanie skryptu publikującego adres URL jako osobnej usługi systemd, również zależnej od `network-online.target`. Podejście to zapewniałoby większą kontrolę nad kolejnością uruchamiania komponentów systemu. Przykładową konfigurację jednostki typu `oneshot` zaprezentowano na listingu 19.

```

1  [Unit]
2  Description=AddiPi Camera URL Publisher
3  After=network-online.target addipi-agent.service
4  Wants=network-online.target
5
6  [Service]
7  Type=oneshot
8  User=pi
9  ExecStart=/home/pi/scripts/publish-camera-url.sh
10 RemainAfterExit=no
11
12 [Install]
13 WantedBy=multi-user.target

```

Listing 19: Alternatywna konfiguracja skryptu publikacji jako usługi systemd.

9.7 Podsumowanie konfiguracji w za brzegowego

W tabeli 12 zestawiono oraz sklasyfikowano wszystkie kluczowe usługi systemowe i zadania harmonogramu skonfigurowane w obrębie mikrokomputera Raspberry Pi.

Tabela 12: Podsumowanie usług i zadań cyklicznych na platformie Raspberry Pi.

Usługa / komponent	Mechanizm mieniowy	uruchomienie	Opis realizowanej odpowiedzialności
<i>OctoPrint</i>	systemd w <i>OctoPi</i>)	(natywny	Niskopoziomowe sterowanie drukarek przez interfejs USB, generowanie lokalnego strumienia wizyjnego.
<i>AddiPi Agent</i>	systemd agent.service)	(addipi-	Utrzymywanie sesji z <i>IoT Hub</i> , obsługa <i>Direct Methods</i> , transmisja danych telemetrycznych.

Tabela 12 – ciąg dalszy z poprzedniej strony

Usługa / komponent	Mechanizm uruchomieniowy	Opis realizowanej odpowiedzialności
<i>Cloudflare Tunnel</i>	cron (@reboot)	Zestawienie szyfrowanego połączenia wychodzącego, inicjowane z opóźnieniem 60 s od startu systemu.
Odnowienie adresu URL	cron (0 */6 * * *)	Iteracyjna renegotiacja sesji tunelu oraz przesyłanie aktualnego rejestru do usługi <i>Video Service</i> .

10 Wdrożenie i infrastruktura produkcyjna

10.1 Hybrydowa strategia architektoniczna

Infrastruktura produkcyjna systemu *AddiPi* bazuje na paradygmacie wielochmurowym (ang. *Multi-Cloud*). Podział odpowiedzialności pomiędzy dostawców zasobów został podyktowany ich specyfikacją techniczną, modelami usługowymi oraz dostępnością bezpłatnych pakietów deweloperskich (ang. *free tier*).

W tabeli 13 zestawiono szczegóły struktur lokowania poszczególnych komponentów systemu, przypisane im platformy sprzętowo-programowe oraz modele dostarczania usług.

Tabela 13: Podział infrastruktury produkcyjnej systemu *AddiPi*.

Platforma / dostawca	Model usługowy	Hostowane komponenty ekosystemu
<i>Microsoft Azure</i>	PaaS / usługi zarządzane	Warstwa danych i integracji: <i>Cosmos DB</i> , <i>Blob Storage</i> , <i>Service Bus</i> oraz <i>IoT Hub</i> .
<i>Oracle Cloud (OCI)</i>	IaaS (Maszyna wirtualna)	Kontenery <i>Docker</i> <i>Compose</i> : <i>Auth</i> , <i>User</i> , <i>Files</i> , <i>Queue</i> oraz <i>Printer Service</i> .
<i>Vercel</i>	Serverless / Jamstack	Aplikacja kliencka (React) oraz bezserwerowa usługa dystrybucji wideo (<i>Video Service</i>).
<i>Raspberry Pi</i>	Lokalny (Brzegowe)	Lokalny sterownik <i>OctoPrint</i> oraz demon <i>AddiPi Agent</i> .

Przyjęta topologia wynika bezpośrednio z limitów platformy *Vercel*, która uniemożliwia egzekucję procesów długotrwale działających (ang. *long-running processes*), takich jak stałe nasuchiwanie kolejki *Service Bus* czy cykliczne zadania orkiestracji. Komponenty wymagające ciągłej aktywności ulokowano na bezpłatnej instancji VM *Standard*.

E2.1.1. Micro w chmurze *Oracle* [22], podczas gdy *Microsoft Azure* dostarcza standaryzowane, w pełni zarządzane interfejsy zapisu oraz wymiany komunikatów.

10.2 Zarządzane usługi integracji danych i komunikacji chmurowej

Wszystkie zasoby dostarczane przez platformę *Microsoft Azure* funkcjonują wyłącznie w modelu PaaS, stanowiąc scentralizowany węzeł wymiany informacji i persystencji danych dla

komponentów uruchomionych w infrastrukturze *Oracle Cloud* oraz na mikrokomputerze brzegowym. W tabeli 14 wyszczególniono parametry wdrożeniowe oraz specyfikację konfiguracji poszczególnych usług chmurowych.

Tabela 14: Specyfikacja konfiguracji usług Microsoft Azure wykorzystanych w projekcie.

Nazwa usługi	Rola architektoniczna	Parametry konfiguracyjne i plany
<i>Azure Cosmos DB</i>	Baza danych NoSQL	Interfejs API SQL, baza addipi, kontenery: users, jobs, refresh-tokens; klucz partycjonowania: /id.
<i>Azure Blob Storage</i>	Repozytorium obiektowe	Instancja addipifiles, kontener gcode, warstwa wydajnościowa Standard LRS.
<i>Azure Service Bus</i>	Broker komunikatów	Przestrzeń nazw addipisb, kolejka print-queue, warstwa Basic, parametr TTL wiadomości: 14 dni.
<i>Azure IoT Hub</i>	Zarządzanie wzami IoT	Hub addipi-iothub, to samo raspberry-pi-mkt-01, plan bezpłatny F1, protokół MQTT.

10.3 Automatyzacja orkiestracji chmurowej — Terraform

Kompletna infrastruktura chmurowa systemu została zdefiniowana w sposób deklaratywny zgodnie z paradygmatem *Infrastructure as Code* (IaC) przy użyciu narzędzia *Terraform* [21]. Podejście to gwarantuje powtarzalność środowisk i umożliwia odtworzenie zasobów na dowolnych kontach dostawców chmurowych. Repozytorium AddiPi-Infrastructure zostało podzielone na dwa autonomiczne moduły:

- terraform/ — konfiguracja zasobów *Microsoft Azure* (grupa zasobów AddiPi-REG, Storage Account, Service Bus, Cosmos DB oraz IoT Hub) z wykorzystaniem dostawcy hashicorp/azurerm w wersji ~> 4.0,
- terraform-oci/ — konfiguracja zasobów *Oracle Cloud* (sieć vcn-addipi, reguły bezpieczeństwa dla portów: 3001, 3002, 3050, 3070, 5000, instancja VM.Standard.E2.1.Micro o parametrach 1 OCPU i 1 GB RAM oraz publiczny adres IP) z użyciem dostawcy oracle/oci w wersji ~> 6.0.

Oba moduły wykorzystują pliki `imports.tf` zawierające deklaratywne bloki `import`, co pozwala na asymilację istniejącej infrastruktury do pliku stanu (ang. *state file*) bez konieczności niszczenia i ponownego tworzenia obiektów chmurowych.

Proces inicjalizacji dla wybranego środowiska (katalog `terraform/` dla Azure lub `terraform-oci/` dla OCI) polega na przygotowaniu parametrów wejściowych w pliku `terraform.tfvars` (na podstawie dostarczonego szablonu `.example`) i wykonaniu sekwencji instrukcji zaprezentowanych na listingu 20.

```
1  # Inicjalizacja providerów i~wdroenie konfiguracji
2  terraform init
3  terraform plan
4  terraform apply
```

Listing 20: Inicjalizacja i aplikacja planu infrastruktury przez Terraform [21].

Po pomyślnym zastosowaniu planu, wygenerowane wartości niejawne (ang. *sensitive outputs*), takie jak parametry poczeniowe czy klucze główne baz danych, pobiera się bezpośrednio z poziomu konsoli za pomocą instrukcji typu `terraform output -raw <nazwa_zmiennej>` (np. dla zmiennych `storage_connection_string`, `service_bus_connection_string`, `cosmos_endpoint` oraz `cosmos_key`). Pozyskane w ten sposób sekrety są następnie wstrzykiwane do pliku konfiguracyjnego `.env` na produkcyjnej maszynie wirtualnej OCI.

10.4 Specyfikacja konteneryzacji i optymalizacja obrazów

W celu zapewnienia powtarzalności środowisk uruchomieniowych, każdy z mikroservisów zostaje odizolowany w strukturze kontenera *Docker* [20]. Dla usług opartych na technologii *Node.js* [8] wdrożono mechanizm wieloetapowego budowania (ang. *multi-stage build*), którego celem jest separacja założeń deweloperskich i procesu kompilacji *TypeScript* (etap *builder*) od lekkiego obrazu produkcyjnego. Wzorców konfiguracji tego potoku dla usługi *Auth Service* zaprezentowano na listingu 21.

```
1  FROM node:20-alpine AS builder
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm ci
5  COPY tsconfig*.json ./
6  COPY src ./src
```

```

7  RUN npm run build
8
9  FROM node:20-alpine
10 WORKDIR /app
11 ENV NODE_ENV=production
12 COPY package*.json ./
13 RUN npm ci --omit=dev
14 COPY --from=builder /app/dist ./dist
15 EXPOSE 3001
16 CMD ["node", "dist/index.js"]

```

Listing 21: Wieloetapowy plik konfiguracyjny Dockerfile serwisu Node.js.

Z kolei usługa *Files Service* (Python) z uwagi na interpretowany charakter kodu wykorzystuje uproszczony, jednoetapowy schemat oparty na dystrybucji minimalistycznej. Struktur tego pliku, uwzględniając optymalizację rozmiaru warstw poprzez blokowanie pamięci podręcznej pip, przedstawiono na listingu 22.

```

1  FROM python:3.13-slim
2  WORKDIR /app
3  COPY requirements.txt .
4  RUN pip install --no-cache-dir -r requirements.txt
5  COPY app.py .
6  COPY config/ ./config/
7  COPY controllers/ ./controllers/
8  COPY routes/ ./routes/
9  COPY middleware/ ./middleware/
10 EXPOSE 5000
11 CMD ["python", "app.py"]

```

Listing 22: Konfiguracja Dockerfile dla usługi Files Service (Python).

10.5 Automatyzacja procesów CI/CD

W celu zautomatyzowania cyklu wydawniczego oprogramowania, każde repozytorium ródowe zostało zintegrowane z usługą *GitHub Actions*. Zdefiniowane potoki automatyzacji (ang. *workflows*) są wyzwalane po zdarzeniu push do gałęzi *main* i realizują następujące etapy:

1. **Kompilacja i budowanie:** Kompilacja kodu ródowego oraz budowanie produkcyjnego obrazu *Docker*.

2. **Tagowanie zasobu:** Nadanie unikalnego tagu rejestru w postaci

`ghcr.io/AddiPii/<nazwa-serwisu>:latest.`

3. **Publikacja artefaktu:** Automatyczna publikacja gotowego obrazu wewnątrz *GitHub Container Registry* (GHCR).

Na produkcyjnej maszynie wirtualnej OCI osadzono autonomiczny komponent *Watchtower* [38]. Narzędzie to cyklicznie sonduje rejestr GHCR, weryfikuje sygnatury obrazów i po wykryciu nowej kompilacji automatycznie pobiera warstwy oraz restartuje odpowiednie kontenery. Podejście to w pełni realizuje założenia paradygmatu ciągłego wdrażania (ang. *Continuous Deployment*), uniezależniając proces aktualizacji produkcyjnej od manualnej interwencji administratora na serwerze.

10.6 Orkiestracja kontenerowa warstwy serwerowej — Docker Compose

Uruchomienie mikroservisów warstwy serwerowej na instancji *Oracle Cloud* realizowane jest przy użyciu narzędzia *Docker* [20] *Compose*. W środowisku produkcyjnym wykorzystano wariant konfiguracyjny `docker-compose.ghcr.yml`, który pobiera skompilowane artefakty bezpośrednio z rejestru kontenerów GHCR. Rozwiązanie to drastycznie minimalizuje obciążenie procesora i pamięci RAM instancji IaaS, eliminując potrzebę lokalnego budowania obrazów. Każdy z komponentów funkcjonuje w odizolowanym środowisku z dedykowanymi politykami restartu `unless-stopped`, a komunikacja wewnętrzna odbywa się w ramach wirtualnej sieci `addipi-network`.

Na listingu 23 przedstawiono reprezentatywny fragment produkcyjnego pliku konfiguracyjnego.

```
1  version: '3.9'
2
3  services:
4    auth:
5      image: ghcr.io/AddiPii/AddiPi-Auth-Service:latest
6      env_file: .env
7      ports:
8        - "3001:3001"
9      restart: unless-stopped
10
```

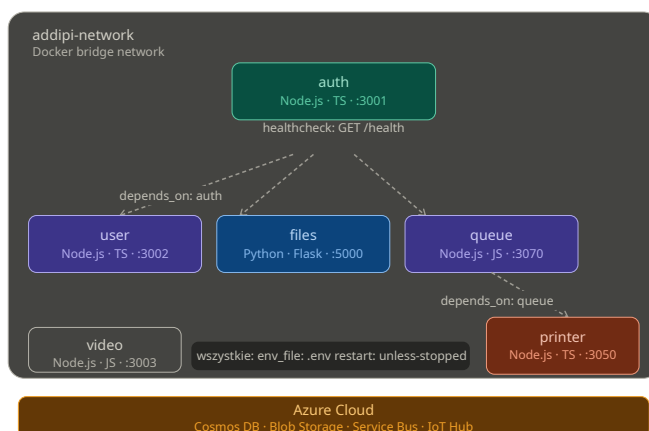
```

11     user:
12       image: ghcr.io/AddiPii/AddiPi-User-Service:latest
13       env_file: .env
14       ports:
15       - "3002:3002"
16       depends_on: [auth]
17       restart: unless-stopped
18
19     printer:
20       image: ghcr.io/AddiPii/AddiPi-Printer-Service:latest
21       env_file: .env
22       ports:
23       - "3050:3050"
24       depends_on: [queue, auth]
25       restart: unless-stopped

```

Listing 23: Fragment produkcyjnego pliku konfiguracyjnego docker-compose.ghcr.yml.

Cigo operacyjn rodowiska zabezpiecza wspomniany wczesniej kontener *Watchtower* [38], dziaajcy w tle jako demon aktualizacyjny. Bezpieczestwo sieciowe instancji maszyn wirtualnych OCI egzekwowane jest na poziomie warstwy sieciowej chmury, ograniczajc ruch przychodzczy wycznie do niezbdnych portów: 22 (SSH), 3001 (*Auth Service*), 3002 (*User Service*), 3050 (*Printer Service*), 3070 (*Queue Service*) oraz 5000 (*Files Service*). Reguy te s definiowane deklaratywnie w module `terraform-oci/`. Topologi tak zorganizowanej sieci kontenerów oraz ich zalenoci startowe przedstawiono na rysunku 17.



Rysunek 17: Diagram architektury kontenerów Docker Compose i ich relacji zalenoci.

10.7 Zarządzanie konfiguracją i sekretami przez zmienne środowiskowe

Konfiguracja serwisów uruchamianych na instancji OCI realizowana jest przy pomocy zmiennych środowiskowych przekazywanych do kontenerów *Docker* [20] za pośrednictwem pliku `.env`. Mechanizm ten umożliwia oddzielenie konfiguracji od kodu źródłowego, co upraszcza wdrażanie aplikacji w różnych środowiskach oraz zwiększa bezpieczeństwo poprzez eliminację konieczności przechowywania poufnych danych w repozytoriach kodu.

W środowisku produkcyjnym plik `.env` udostępniany jest wszystkim serwisom za pomocą dyrektywy `env_file` w konfiguracji Docker Compose. Repozytorium projektu zawiera również plik `.env.example`, stanowiący szablon wymaganej konfiguracji i dokumentujący znaczenie poszczególnych parametrów.

Tabela 15 przedstawia najważniejsze zmienne środowiskowe wykorzystywane przez system.

Tabela 15: Kluczowe zmienne środowiskowe systemu

Zmienna	Przeznaczenie
COSMOS_ENDPOINT	Adres sieciowy instancji bazy danych.
COSMOS_KEY	Główny klucz autoryzacji i odczytu/zapisu.
STORAGE_CONN	Parametry połączeniowe repozytorium obiektów.
SERVICE_BUS_CONN	Ciąg połączeniowy brokera komunikatów.
IOT_HUB_SERVICE_CS	Interfejs usługowy do wywołania metod RPC.
IOT_HUB_EVENT_HUB_CS	Punkt odbioru asynchronicznej telemetrii.
JWT_SECRET	Klucz podpisywania tokenów dostępu (<i>access</i>).
JWT_REFRESH_SECRET	Klucz podpisywania tokenów odwołania (<i>refresh</i>).
AUTH_SERVICE_URL	Adres bazowy usługi uwierzytelniania
EMAIL_USER	Konto nadawcze komunikatów aktywacyjnych.
EMAIL_PASSWORD	Hasło aplikacyjne dedykowanej skrzynki e-mail.
DEVICE_CONNECTION_STRING	Token uwierzytelniania urządzenia zarejestrowanego w <i>Azure IoT Hub</i>
OCTOPRINT_API_KEY	Klucz API platformy <i>OctoPrint</i>

10.8 Wdrożenie warstwy prezentacji oraz usługi wideo

Aplikacja kliencka (frontendowa) oraz usługa *Video Service* zostały wdrożone na platformie chmurowej *Vercel* [29], co zapewnia ich globalną dystrybucję za pośrednictwem sieci CDN (ang. *Content Delivery Network*). Proces budowania i aktualizacji zasobów statycznych pod adresem <https://addipi.vercel.app> realizowany jest w pełni automatycznie w modelu

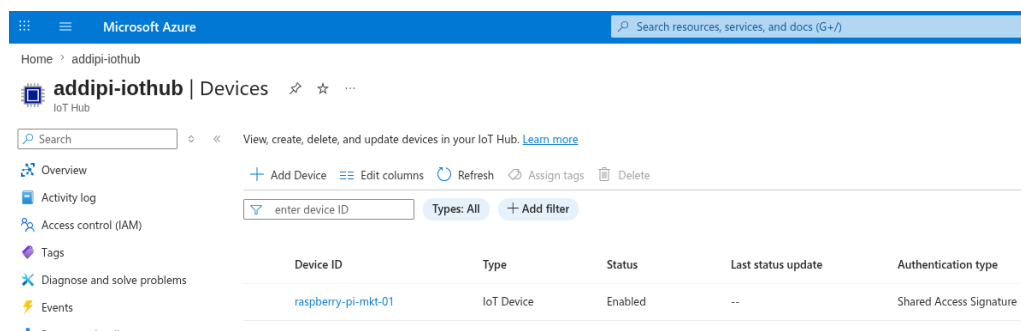
CD po każdym zdarzeniu push do gązi main repozytorium *GitHub*, uruchamiając komendę produkcyjną środowiska *Vite* [17] `npm run build`.

Specyfika platformy *Vercel* opiera się na paradygmacie bezserwerowym (ang. *Serverless*), który narzuca sztywne ograniczenia czasowe na egzekucję kodu (ang. *execution timeouts*). Z tego względu infrastruktura ta została wykorzystana wyłącznie do hostowania interfejsu użytkownika oraz lekkiej, bezstanowej usługi *Video Service*, przechowującej adres URL strumienia w pamięci RAM. Mikroserwisy wymagające ciągłego, asynchronicznego nasłuchiwania kolejek komunikatów (*Queue Service*) oraz orkiestracji zadań telemetrycznych (*Printer Service*) charakteryzują się naturą procesów długotrwale działających (ang. *long-running processes*). Ich implementacja w modelu *Serverless* byłaby niemożliwa, co w pełni uzasadnia ich lokowanie na dedykowanej maszynie wirtualnej OCI.

10.9 Wdrożenie komponentu brzegowego

Wdrożenie komponentu *AddiPi Agent* zrealizowano bezpośrednio w środowisku lokalnym mikrokomputera Raspberry Pi. Zgodnie ze szczegółowym opisem zawartym w rozdziale 9, proces ten oparto na demonie systemowym `systemd` (`addipi-agent.service`). Konfiguracja ta gwarantuje pełną autonomię wdrażania brzegowego, realizując automatyczny rozruch skryptów po starcie zasilania oraz deterministyczne wznowianie pracy w przypadku awarii procesu. Zapewnia to bezobsługową i niezawodną integrację lokalnego sterownika *OctoPrint* z usługami chmurowymi.

Bezpośrednie, produkcyjne potwierdzenie poprawnej rejestracji oraz aktywnego stanu poczenia urządzenia brzegowego w chmurze obliczeniowej zaprezentowano na rysunku 18, przedstawiającym panel administracyjny usługi *Azure IoT Hub*.



The screenshot shows the Microsoft Azure portal interface for an IoT Hub named 'addipi-iotHub'. The 'Devices' section is active, displaying a table with one device: 'raspberrypi-mkt-01'. The table columns are Device ID, Type, Status, Last status update, and Authentication type. The device is of type 'IoT Device', status is 'Enabled', and uses 'Shared Access Signature' authentication.

Device ID	Type	Status	Last status update	Authentication type
raspberrypi-mkt-01	IoT Device	Enabled	--	Shared Access Signature

Rysunek 18: Widok zarejestrowanego i poczonego mikrokomputera w panelu usługi Azure IoT Hub.

11 Podsumowanie i wnioski końcowe

11.1 Osiągnięte rezultaty i wkład inżynierski

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie, implementacja oraz walidacja kompletnego, rozproszonego systemu umożliwiającego zdalne i wielodostępne zarządzanie procesami druku 3D z wykorzystaniem zaawansowanych technologii chmurowych, paradygmatu mikroserwisów oraz platformy sprzętowej Raspberry Pi pełniące rolę lokalnego agenta orkiestracji. Zakończony cel badawczo-projektowy został w pełni osiągnięty poprzez wytworzenie, wdrożenie i przetestowanie ekosystemu *AddiPi*, który w sposób bezszwowy integruje warstwę sprzętową, oprogramowanie brzegowe, aplikacje webowe oraz rozproszoną infrastrukturę chmurową.

W ramach realizacji projektu inżynierskiego opracowano i zaimplementowano następujące komponenty oraz mechanizmy:

- **Rozproszona architektura mikroserwisowa** [14] [28] — struktura warstwy serwerowej złożona z szeregu autonomicznych serwisów komunikujących się hybrydowo za pośrednictwem synchronicznych interfejsów REST API oraz asynchronicznej magistrali komunikatów *Azure Service Bus*,
- **Scentralizowany podsystem bezpieczeństwa** — model uwierzytelniania i autoryzacji bazujący na standardzie podwójnych tokenów JWT [2], integrujący mechanizm bezpiecznego odwołania sesji, dwuetapową weryfikację adresów e-mail w domenie akademickiej oraz kontrolę dostępu opartą na rolach (RBAC),
- **Mechanizm orkiestracji i harmonogramowania zadań** — zaawansowany algorytm zarządzania kolejką produkcyjną, pozwalający zarówno na natychmiastową alokację zasobów sprzętowych, jak i długoterminowe planowanie kalendarzowe procesów przyrostowych,
- **Integracja brzegowa IoT** — powiązanie platformy sprzętowej z usługą *Azure IoT Hub* [15], umożliwiające synchroniczne sterowanie drukarkami 3D za pomocą wywołania metod bezpośrednich (ang. *Direct Methods*) w standardzie RPC,
- **System monitorowania w czasie rzeczywistym** — mechanizm akwizycji danych

telemetrycznych urządzenia (metryki termiczne i statusy mechaniczne) oraz asynchronicznej dystrybucji strumienia wideo z kamery CSI w formacie MJPEG,

- **Wielojzyczna aplikacja kliencka** — responsywna warstwa prezentacji zbudowana w oparciu o bibliotek *React* [16] oraz język TypeScript, wspierająca architekturę dynamicznych motywów graficznych (jasny i ciemny) oraz wzorzec projektowy RWD,
- **Infrastruktura jako kod (IaC)** — deklaratywne szablony konfiguracyjne narzędzia *Terraform* [21], automatyzujące procesy apro wizacji i replikacji zasobów chmurowych w architekturze hybrydowej *Microsoft Azure* oraz *Oracle Cloud Infrastructure* [22],
- **Zautomatyzowany potok wdrożeniowy (CI/CD)** — konfiguracja środowisk potokowych *GitHub Actions* zintegrowanych z rejestrem GHCR oraz automatycznym demonem aktualizacyjnym *Watchtower* [38] w produkcyjnym środowisku kontenerowym,
- **Projekt mechaniczny CAD** — zoptymalizowana pod kątem tolerancji wymiarowych obudowa mikrokomputera oparta na modelu otwartym *GrabCAD* [35] wraz z przegubowym ramieniem pozycjonującym kamer, zaprojektowana w programie *SolidWorks* [23] i przygotowana do wytwarzania przyrostowego (FDM),
- **Automatyzacja i konfiguracja w środowisku brzegowym** — autorskie skrypty powłoki Bash zintegrowane z demonem *systemd* oraz systemowym harmonogramem *cron*, zabezpieczające bezobsługowy rozruch sieciowy i stabilność tunelu *Cloudflare Tunnel* [37].

Ostatecznie w ramach pracy powstał kompletny, w pełni funkcjonalny prototyp rozproszonego systemu zdalnego zarządzania autonomicznym stanowiskiem druku trójwymiarowego. Gotowe rozwiązanie zostało pomyślnie wdrożone i poddane długoterminowej walidacji w rzeczywistym środowisku pracy z fizyczną drukarką 3D. Przeprowadzone testy integracyjne oraz użytkowe potwierdziły wysoką niezawodność, bezpieczeństwo oraz bezobsługowość całego ekosystemu, udowadniając poprawność przyjętych założeń inżynierskich oraz wysoką efektywność wybranych technologii chmurowych i brzegowych.

11.2 Analiza napotkanych trudności projektowo-implementacyjnych

Realizacja rozproszonego systemu *AddiPi* wiązała się z koniecznością identyfikacji i rozwiązania szeregu zaawansowanych problemów technicznych. Wynikały one zarówno z heterogenicznego charakteru architektury wielochmurowej, jak i z fizycznej integracji zewnętrznych usług chmurowych z urządzeniami brzegowymi Internetu Rzeczy (IoT).

- **Integracja rozproszonego i heterogenicznego środowiska chmurowego**

Jednym z największych wyzwań architektonicznych było zapewnienie stabilnej orkiestracji komponentów działających w odmiennych środowiskach wykonawczych [28]. Ekosystem wykorzystuje połączenie zasoby *Microsoft Azure* (*Cosmos DB*, *Blob Storage*, *Service Bus*, *IoT Hub*), instancji maszyn wirtualnych *Oracle Cloud Infrastructure* [22], platform *Vercel* [29] oraz fizyczny mikrokomputer brzegowy działający wewnątrz sieci lokalnej. Konieczne było zaprojektowanie spójnego modelu komunikacji, który uwzględniałby fluktuacje sieciowe, opóźnienia pakietów [3] oraz skrajnie odmienne standardy uwierzytelniania dostawców. Wyzwanie to zminimalizowano poprzez wdrożenie hybrydowej architektury opartej na synchronicznych interfejsach REST API oraz asynchronicznej wymianie komunikatów z wykorzystaniem kolejki brokera, uzupełnionej o autorskie mechanizmy ponawiania danych.

- **Spójność i normalizacja reprezentacji czasu**

Poszczególne warstwy systemu natywnie operowały na zróżnicowanych standardach zapisu dat i czasu. Informacje generowane przez silnik bazy danych, usługi backendowe oraz biblioteki frontendowe nie zawsze cechowały się identyczną strukturą, co generowało błąd w interpretacji i renderowaniu harmonogramów zadań. Problem ten rozwiązano na poziomie warstwy prezentacji poprzez implementację scentralizowanego modułu normalizacji danych chronologicznych, wykorzystującego autorską funkcję `normalizeDateString`.

- **Koordinacja sekwencyjności startu mikroservisów**

W fazie wdrażania środowiska kontenerowego zidentyfikowano, że natywna dyrektywa `depends_on` w konfiguracji *Docker Compose* gwarantuje wyłącznie kolejność inicjalizacji procesów, nie weryfikując rzeczywistej gotowości operacyjnej API do obsługi danych sieciowych. Prowadziło to do błędów początkowo w sytuacji, gdy serwisy zależne próbowały komunikować się z komponentami będącymi jeszcze w fazie wewnętrznego

rozruchu. Wyzwanie to zaadresowano poprzez zaimplementowanie mechanizmów diagnostycznych stanu działania (ang. *health checks*) oraz procedur ponawiania prób z wykładniczym czasem oczekiwania (ang. *retry with exponential backoff*) po stronie klientów HTTP.

- **Kompatybilno systemów modułów CommonJS oraz ESM**

Podczas integracji warstwy serwerowej z bibliotekami SDK *Azure IoT* w środowisku Node.js napotkano na konflikty struktur modułów *CommonJS* oraz *ECMAScript Modules* (ESM). Wybrane pakiety załane eksportowały swoje składowe w sposób silnie uzależniony od kontekstu uruchomieniowego. W celu zapewnienia pełnej kompatybilności i stabilności kodu, w strukturze usługi *Printer Service* zaimplementowano mechanizm defensywnej detekcji i dynamicznej ekstrakcji dostępnych interfejsów eksportu.

- **Udostępnienie strumienia wideo poza barier sieci lokalnej**

Ponieważ serwer *OctoPrint* operuje wewnątrz lokalnej sieci prywatnej, strumień wizyjny MJPEG nie był domyślnie trasowany do publicznego Internetu, co uniemożliwiało jego odbiór przez aplikację webową. Dodatkowo barierą stanowiła zmienność adresów URL przy każdej renegocjacji połączenia tunelowego. Wyzwanie to rozwiązano za pomocą technologii *Cloudflare Tunnel* [37] połączonej z autonomicznymi usługami *Video Service*, która działa jako dynamiczny rejestrator aktualnego punktu docelowego wideo.

- **Obsługa opóźnień w synchronicznej komunikacji IoT**

Sterowanie parametrami fizycznej maszyny przy użyciu metod bezpośrednich (ang. *Direct Methods*) generowało opóźnienia rzędu kilku sekund w warunkach gorszej przepływności sieciowej w związku z brzegowym. Zjawisko to zostało uwzględnione na poziomie projektowania interfejsu użytkownika (UX) poprzez wprowadzenie asynchronicznych stanów ładowania, blokad przycisków przed wielokrotnym kliknięciem (ang. *debouncers*) oraz deterministycznej obsługi przekroczenia czasu oczekiwania (ang. *timeouts*).

Pomimo opisanych trudności, wszystkie zidentyfikowane bariery technologiczne zostały pomyślnie wyeliminowane na etapie implementacji. Pozwoliło to na uzyskanie w pełni stabilnego, rozproszonego systemu, który w sposób bezwzględny spełnia wszystkie założenia inżynierskie określone w wymaganiach projektu.

11.3 Ograniczenia przyjętego rozwiązania

Pomimo pełnego spełnienia założonych wymagań projektowych, opracowany system posiada kilka ograniczeń technicznych wynikających z przyjętych kompromisów architektonicznych oraz specyfiki wybranych technologii.

- **Skala stanowiska produkcyjnego** — system został zoptymalizowany pod kątem obsługi pojedynczego urządzenia. Choć architektura pozwala na skalowanie, obecna implementacja dedykowana jest dla jednej kolejki zadań i jednego aktywnego węzła brzegowego.
- **Zależność od infrastruktury zewnętrznej** — podgląd wideo opiera się na bezpłatnym wariantcie `trycloudflare.com`, co uniemożliwia przypisanie statycznego adresu URL i może generować okresowe przerwy w dostępności usługi sieciowej.
- **Limity darmowych planów chmurowych** — wdrożenie produkcyjne bazuje na bezpłatnych pakietach zasobów (*Oracle Always Free* oraz *Azure Free Tier*). Posiadają one sztywne limity wydajnościowe i pojemnościowe, wymagające migracji do planów komercyjnych przy wyższym obciążeniu.
- **Punkt pojedynczej awarii (SPOF) [3]** — komunikacja z urządzeniem zależy bezpośrednio od stabilności serwera *OctoPrint* oraz mikrokomputera. Awaria warstwy brzegowej całkowicie paraliżuje proces realizacji nowych zleceń.
- **Brak statycznej walidacji kodu sterującego** — system nie realizuje automatycznej weryfikacji poprawności struktury plików G-code przed alokacją w kolejce. Walidacja poprawności technologicznej pliku spoczywa wyłącznie na operatorze.
- **Ograniczone pokrycie testami** — bieżąca wersja prototypowa nie została wyposażona w kompleksowy, zautomatyzowany zestaw testów integracyjnych typu *end-to-end*, który pokrywałby pełen cykl życia zadania.

11.4 Perspektywy i potencjalne kierunki dalszego rozwoju systemu

Opracowany system *AddiPi* został zaprojektowany z zachowaniem paradygmatu modularności oraz separacji odpowiedzialności, co zapewnia wysoką elastyczność i podatność na dalsze

rozbudow — zarówno w obszarze funkcjonalnym, jak i architektonicznym. Jako kluczowe, perspektywiczne kierunki rozwoju platformy zidentyfikowano następujące zagadnienia:

- **Obsługa środowiska wielourzędzeniowego (farma drukarek)** — rozszerzenie obecnej struktury dokumentów JSON o unikalne identyfikatory fizycznych maszyn oraz powiązanie z nimi odrębnych, izolowanych potoków kolejkowych, co umożliwi scentralizowane zarządzanie rozproszonym parkiem maszynowym z poziomu pojedynczej instancji aplikacji,
- **Autonomiczny system powiadomień natychmiastowych** — integracja architektury sieciowej z technologiami *Web Push* lub infrastruktur *Firebase Cloud Messaging* (FCM) w celu natywnego informowania operatorów o zmianach stanów zadania (rozpoczęcie, sukces, awaria) bez konieczności utrzymywania aktywnej sesji w przeglądarce,
- **Statyczna analiza plików G-code przed wydrukiem** — implementacja modułu analizy skadniowej kodu sterującego na etapie jego przesyłania do *Files Service*, co pozwoliłoby na precyzyjne, niezależne szacowanie czasu druku, wolumetrycznego zużycia filamentu oraz automatyczne wykrywanie błędów konfiguracji slicera,
- **Detekcja anomalii produkcyjnych z wykorzystaniem sztucznej inteligencji** — integracja strumienia wideo MJPEG z modelami uczenia maszynowego (np. konwolucyjnymi sieciami neuronowymi) w celu automatycznego wykrywania krytycznych wad druku w czasie rzeczywistym, takich jak odklejenie warstwy bazowej od stołu, zjawisko utraty spójności (ang. *spaghetti failure*) czy zablokowanie ekstrudera,
- **Rozbudowa mechanizmów federacyjnego uwierzytelniania** — implementacja protokołów *OAuth 2.0* oraz *OpenID Connect* (OIDC), co umożliwi bezpieczną integrację z zewnętrznymi dostawcami tożsamości (ang. *Identity Providers*), takimi jak konta *Microsoft*, *Google* lub *GitHub*,
- **Dedykowana aplikacja mobilna** — implementacja natywnej lub hybrydowej warstwy prezentacji (np. w technologii *React Native*), co podniosłoby ergonomię monitorowania telemetrii urządzenia na ekranach przenośnych,
- **Skalowanie i orkiestracja infrastruktury produkcyjnej** — przeniesienie struktury kontenerowej do środowiska orkiestracji *Kubernetes* (K8s) w celu wdrożenia mecha-

nizmów automatycznego skalowania pionowego i poziomego, zaawansowanego równoważenia obciążenia (ang. *load balancing*) oraz scentralizowanego routingu usług,

- **Rozszerzenie warstwy jakości i odporności oprogramowania** — uzupełnienie kodu źródłowego o pełne pokrycie testami automatycznymi [7], generowanie wyżej dokumentacji API w standardzie *OpenAPI/Swagger* oraz wdrożenie mechanizmów ochrony przed przeciążeniem interfejsów, takich jak *rate limiting* czy warstwa buforowania *Redis*.

11.5 Wnioski końcowe

Przeprowadzone prace projektowe, implementacyjne oraz walidacyjne potwierdziły, że integracja rozproszonej architektury mikroserwisowej z usługami chmurowymi w modelu PaaS (*Platform as a Service*) stanowi wysoce efektywny paradygmat budowy systemów zarządzania autonomicznymi stanowiskami druku 3D. Wszystkie założone we wstępie cele pracy zostały w pełni zrealizowane. Opracowany system *AddiPi* umożliwia w pełni bezpieczne, zdalne przesyłanie plików z kodem sterującym, elastyczne harmonogramowanie zadań, ciągłe monitorowanie parametrów środowiskowych procesu oraz orkiestrację urządzeń z poziomu uniwersalnego interfejsu webowego.

Zastosowanie architektury zorientowanej na mikroserwisy [28] okazało się właściwym i uzasadnionym wyborem projektowym dla systemu tej klasy. Dekompozycja funkcjonalna na niezależne komponenty uproszczyła rozwój poszczególnych warstw logicznych, zwiększyła ogólną modularność rozwiązania oraz umożliwia bezprzerwowe, asynchroniczne wdrażanie aktualizacji (ang. *zero-downtime deployment*). Jednocześnie realizacja projektu wykazała, że podejście mikroserwisowe generuje dodatkowy narzut architektoniczny (ang. *overhead*) [3] [14], związany z koniecznością zapewnienia spójności danych, scentralizowanego zarządzania konfiguracją środowisków oraz monitorowania rozproszonych potoków komunikacyjnych. Kluczowym czynnikiem sukcesu było również rygorystyczne stosowanie zasad czystego kodu [7], co umożliwiło sprawną integrację heterogenicznych technologii (TypeScript [6], Python [9]) w ramach jednego ekosystemu.

Wykorzystanie ekosystemu *Microsoft Azure* pozwoliło na sprawną implementację kluczowych mechanizmów platformy bez konieczności ponoszenia kosztów budowy i utrzymania stacjonarnej infrastruktury serwerowej (ang. *on-premise backend*). Krytyczne znaczenie dla stabilności systemu miało wdrożenie usług *Azure IoT Hub*, *Azure Service Bus* [13] oraz

Azure Cosmos DB [11]. Komponenty te umożliwiają bezproblemową realizację asynchronicznego routingu komunikatów, synchronicznego sterowania urządzeniem brzegowym przez protokół RPC oraz bezpiecznej persistencji obiektów JSON.

Realizacja warstwy sprzętowej potwierdza, że mikrokomputer Raspberry Pi zintegrowany z serwerem *OctoPrint* [5] stanowi stabilną, skalowalną i elastyczną platformę pośredniczącą pomiędzy fizyczną kinematyką drukarki 3D a systemami rozproszonymi. Opracowany komponent *AddiPi Agent* zabezpiecza poprawną i dwukierunkową wymianę informacji, gwarantując zarówno niskopoziomowe sterowanie maszyn, jak i ciągłą akwizycję danych telemetrycznych w czasie rzeczywistym.

Uzyskane rezultaty aplikacyjne wskazują, że system *AddiPi* posiada wysoki potencjał wdrożeniowy i może znaleźć realne zastosowanie nie tylko w strukturach akademickich laboratoriów studenckich, lecz również w placówkach badawczo-rozwojowych, szkołach technicznych, biurach projektowych oraz przedsiębiorstwach wykorzystujących współdzielone zasoby wytwarzania przyrostowego. Modułowa topologia rozwiązania pozostawia pełną swobodę w zakresie dalszej ewolucji systemu, umożliwiając implementację nowych funkcji oraz orkiestrację większej liczby maszyn bez konieczności modyfikacji lub przebudowy istniejących komponentów rdzenia.

Literatura

- [1] Ian Gibson, David Rosen, Brent Stucker, and Mahyar Khorasani. *Additive Manufacturing Technologies*. Springer Nature, 3 edition, 2021.
- [2] M. Jones, J. Bradley, and N. Sakimura. RFC 7519: JSON Web Token (JWT). Technical report, IETF, 2015.
- [3] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [4] Niels Provos and David Mazières. A Future-Adaptable Password Scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference*, 1999.
- [5] Gina Häußge. Octoprint – The snappy web interface for your 3D printer. <https://octoprint.org>. Dostp: 2026.
- [6] Microsoft Corporation. Typescript – JavaScript With Syntax For Types. <https://www.typescriptlang.org>. Dostp: 2026.
- [7] Robert C. Martin. *Czysty kod. Podręcznik dobrego programisty*. Helion, 2 edition, 2022.
- [8] OpenJS Foundation. Node.js – JavaScript runtime built on Chrome's V8 engine. <https://nodejs.org>. Dostp: 2026.
- [9] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>. Dostp: 2026.
- [10] OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>. Dostp: 2026.
- [11] Microsoft Corporation. Azure Cosmos DB documentation. <https://learn.microsoft.com/en-us/azure/cosmos-db/>. Dostp: 2026.
- [12] Microsoft Corporation. Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>. Dostp: 2026.
- [13] Microsoft Corporation. Azure Service Bus documentation. <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>. Dostp: 2026.

- [14] Chris Richardson. *Mikroserwisy. Wzorce z przykladami w jzyku Java*. Wydawnictwo Naukowe PWN, 2020.
- [15] Microsoft Corporation. Azure IoT Hub documentation. <https://learn.microsoft.com/en-us/azure/iot-hub/>. Dostp: 2026.
- [16] Meta Platforms, Inc. React – The library for web and native user interfaces. <https://react.dev>. Dostp: 2026.
- [17] Evan You et al. Vite – Next Generation Frontend Tooling. <https://vitejs.dev>. Dostp: 2026.
- [18] Poimandres. Zustand – Bear necessities for state management in React. <https://github.com/pmndrs/zustand>. Dostp: 2026.
- [19] Tailwind Labs Inc. Tailwind CSS – Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com>. Dostp: 2026.
- [20] Docker Inc. Docker Documentation. <https://docs.docker.com>. Dostp: 2026.
- [21] HashiCorp. Terraform infrastructure as code. <https://www.terraform.io>. Dostp: 2026.
- [22] Oracle Corporation. Oracle cloud infrastructure free tier. <https://www.oracle.com/cloud/free/>. Dostp: 2026.
- [23] Dassault Systèmes. SOLIDWORKS – 3D CAD Design Software. <https://www.solidworks.com>. Dostp: 2026.
- [24] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>. Dostp: 2026.
- [25] MongoDB, Inc. Mongoddb atlas cloud database service. <https://www.mongodb.com/docs/atlas/index/>. Dostp: 2026.
- [26] Broadcom. Rabbitmq documentation. <https://www.rabbitmq.com/docs>. Dostp: 2026.
- [27] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/43/getting-started/introduction/>. Dostp: 2026.

- [28] Sam Newman. *Budowanie mikrouslug. Projektowanie drobnoziarnistych systemów. Wydanie II*. Helion, 2022.
- [29] Vercel Inc. Vercel build and deploy the best web experiences with the frontend cloud. <https://vercel.com>. Dostp: 2026.
- [30] Andris Reinman. Nodemailer documentation. <https://nodemailer.com/>. Dostp: 2026.
- [31] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/en/stable/>. Dostp: 2026.
- [32] Remix Software. React router documentation. <https://reactrouter.com/>. Dostp: 2026.
- [33] Axios Team. Axios http client documentation. <https://axios-http.com>. Dostp: 2026.
- [34] i18next Team. i18next internationalization framework documentation. <https://i18next.com>. Dostp: 2026.
- [35] 219 Design. Case for Raspberry Pi 4 GrabCAD Library. <https://grabcad.com/library/case-for-raspberry-pi-4-1>. Licencja CC BY 4.0, Dostp: 2026.
- [36] Guy Sheffer et al. Octopi octoprint on raspberry pi. <https://github.com/guysoft/OctoPi>. Dostp: 2026.
- [37] Cloudflare Inc. Cloudflare Tunnel Securely connect your infrastructure. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>. Dostp: 2026.
- [38] containrrr. Watchtower automatically update docker containers. <https://containrrr.dev/watchtower/>. Dostp: 2026.